Учебная практика ПМ.07

# Задание и методические рекомендации

## Выберите свой маршрут

Найдите себя в списке группы и откройте вариант с тем же номером в отдельном файле «Список предметных областей». Из шести маршрутов выполните один по ОС и СУБД из карточки.

MySQL 8 · Linux

MariaDB 10 · Linux

PostgreSQL 15/16 · Linux

SQL Server Express · Windows

MySQL 8 · Windows

PostgreSQL 16 · Windows

ПМ.07 / Соадминистрирование

# Полное задание

1

## Какой результат подготовить

На выделенном учебном сервере выполните базовые операции администратора: подключение, разграничение доступа, ручную резервную копию, восстановление отдельной проверочной базы и запуск одной операции по расписанию. Операция автоматизации, ОС и СУБД указаны в карточке вашего варианта. Один вариант — один рабочий маршрут.

2

## С чего начать

Найдите себя в списке своей группы. Откройте вариант с тем же номером в отдельном файле «Список предметных областей». По ОС и СУБД из карточки откройте один маршрут. Скачайте SQL-файл своего варианта и поместите его на учебную машину. Для подключения используйте выданный паспорт среды.

3

## Что должно быть выдано до начала

Нужен отдельный учебный сервер или виртуальная машина с уже работающей СУБД своего варианта, консольным и графическим клиентами. Преподаватель передаёт адрес, порт, имя начального администратора сервера и способ входа, доступ к терминалу ОС, разрешённый каталог работы и способ переноса файлов. Для Windows дополнительно нужны пути к клиентским утилитам и учётная запись запуска планировщика. Для Linux — имя службы, доступность cron, тип и доступ к журналу его запусков. Для всех сред указывается часовой пояс сервера и планировщика. Запишите эти сведения в исходные условия без паролей.

Установка ОС и СУБД не входит в обязательный маршрут этой методички. Если доступа или параметров подключения ещё нет, сообщите преподавателю номер варианта, ОС и СУБД и запросите доступ к подготовленной среде. До получения доступа можно скачать свой SQL-файл и открыть нужный маршрут. Команды выполняйте только в выданной учебной среде.

4

## Какие данные использовать

В карточке своего варианта скачайте его SQL-файл с тремя учебными таблицами и контрольными строками. Этот файл уже задаёт структуру: проектировать ER-модель повторно не требуется. У каждого варианта своя база `pm07_vNN`. Таблица `access_probe` служит только для проверки прав. Числа, даты и имена в примере учебные; они не являются данными действующей организации.

Для первого разбора дан отдельный SQL-образец «Учёт оборудования лаборатории». Он объясняет действие команд. Обязательный результат — собственный вариант; вторую демонстрационную работу сдавать не нужно. Не требуется выполнять все шесть маршрутов или все тридцать вариантов.

5

## В каком порядке работать

1. Выберите маршрут по ОС и СУБД своего варианта. Проверьте адрес, порт, версию сервера и текущее имя пользователя.
2. Под начальным администратором создайте учебную базу и три учётные записи. Пользователь `admin` управляет своей базой, `app_user` работает с данными, `readonly` только читает. Создание серверных учётных записей выполняется из-под начального администратора, а не из-под `admin` с правами только на одну базу.
3. Загрузите готовый SQL своего варианта. Проверьте разрешённые и запрещённые действия отдельными подключениями. Запишите точную причину отказа.
4. Сделайте ручную резервную копию, сохраните SHA-256 и контрольные значения. Восстановите **отдельную** базу `pm07_vNN_restore`. Исходная база при этой проверке сохраняется.
5. Подготовьте один скрипт операции своей карточки. Сначала проверьте его вручную, затем назначьте ближайший однократный запуск и только после этого постоянное расписание.
6. Сохраните журнал, фактические результаты проверок, скрипты и параметры запуска. Объясните одно индивидуальное дополнение, которого ещё не было в исходном образце.

6

### Что проверить и сдать

Подтвердите подключение из CLI и графического клиента, матрицу прав, ручной бэкап, восстановление отдельной копии и результат назначенной операции. Для таблиц сравните количество строк и контрольную сумму числового поля; один ненулевой размер архива этого не подтверждает. SHA-256 проверяет неизменность файла, а не полноту восстановленных данных.

Откройте в ZIP из карточки практики файл **«Отчёт о выполнении проекта.docx»**. Заполните исходные условия, выполнение и проверку, итоговую проверку и сдачу. В форме есть места для результатов подключения, проверки прав, восстановления, своей операции, отказа и расписания, дополнения и передачи. Вставляйте собственные результаты и рисунки с подписями.

Сохраните SQL настройки прав, скрипт автоматизации, журнал, контрольные значения до и после восстановления, резервную копию и параметры расписания. Подготовьте репозиторий и сдачу по шагам. Остальные формы возьмите из карточки своей группы.

ПМ.07 / Соадминистрирование

# Файлы и выбор маршрута

7

- Методические рекомендации, PDF.
- Отдельный список предметных областей, PDF.
- Бланк отчёта, DOCX. На сайте файл скачивается как «Отчет о выполнении.docx». В ZIP карточки этот же бланк называется «Отчёт о выполнении проекта.docx».
- Тридцать карточек вариантов и собственные SQL-файлы.
- SQL-образец для MySQL, MariaDB, PostgreSQL, SQL Server.
- Выберите один маршрут. Указанные SQL-файлы — исходные данные, не готовые скрипты автоматизации.

ПМ.07 / Соадминистрирование

# Как вести журнал

8

Для каждого существенного действия записывайте: дату и время, ОС/клиент, учётную запись, базу, выполненную команду, ожидаемый результат, фактический результат и вывод. Сообщение о запрете — успешная проверка прав только тогда, когда отклонена нужная операция. Ошибка пароля, синтаксиса или отсутствующего файла не подтверждает разграничение доступа.

Панель СУБД и консоль ОС — разные места выполнения. SQL вводится в клиенте выбранной СУБД; команды Bash — в терминале Linux; PowerShell — в окне Windows PowerShell. Метки приглашения `mysql>`, `postgres=#`, `PS>` и `$` в команду не копируются.

ПМ.07 / Соадминистрирование

# До первого действия

1

Откройте в отдельном списке областей вариант с вашим номером из списка группы. Выберите маршрут по **обоим** значениям из карточки: СУБД и ОС. Для Ubuntu Server и Debian используйте Linux-маршрут соответствующей СУБД. Версия сервера должна совпадать с карточкой.

2

## · продолжение

| Маршрут | Когда использовать | Первое действие |
|---|---|---|
| MySQL 8 · Linux | Варианты 01, 02, 03, 11, 12, 15, 26, 30 | Откройте этот маршрут и заполните его исходные параметры. |
| MariaDB 10 · Linux | Варианты 04, 05, 06, 13, 17, 24, 29 | Откройте этот маршрут и заполните его исходные параметры. |
| PostgreSQL 15/16 · Linux | Варианты 07, 08, 09, 10, 14, 16, 25, 27 | Откройте этот маршрут и заполните его исходные параметры. |
| SQL Server Express · Windows | Варианты 18, 19, 20, 21, 28 | Откройте этот маршрут и заполните его исходные параметры. |
| MySQL 8 · Windows | Варианты 22 | Откройте этот маршрут и заполните его исходные параметры. |
| PostgreSQL 16 · Windows | Варианты 23 | Откройте этот маршрут и заполните его исходные параметры. |

ПМ.07 / Соадминистрирование

# Правило выбора

3

Не выбирайте СУБД по знакомому названию команды. Например, `mariadb-dump` относится к MariaDB, а `pg_restore` — к архивам PostgreSQL. SQL Server Express использует Планировщик заданий Windows; SQL Server Agent для этого маршрута не требуется.

ПМ.07 / Соадминистрирование

# С чего начать в своём проекте

4

Создайте рабочую папку первым блоком своего маршрута. Выполните команды в терминале учебной машины, куда вошли по паспорту среды. Этот терминал может быть открыт у вас на экране, но его файлы лежат на учебной машине. Проверьте `pwd` в Linux или `Get-Location` в PowerShell. Нужна папка `pm07` в домашнем каталоге Linux или `C:\Practice07` на учебной Windows.

5

### Получите адрес SQL своего варианта

В файле «Список предметных областей» откройте ссылку «Скачайте SQL только варианта ...» из своей карточки. На открывшейся странице скопируйте адрес из строки браузера. Для варианта 05 адрес заканчивается на `variant-05.sql`. Если браузер скачал файл сразу, воспользуйтесь переносом готового файла ниже.

6

## Если учебная машина может скачать файл сама

В терминале учебной Linux-машины выполните `cd "$HOME/pm07"`. Отдельно выполните `read -r -p 'Адрес ссылки на SQL: ' sql_url`. Когда появится запрос, вставьте адрес и нажмите Enter. Затем выполните следующий блок.

```
curl -fL -o input.sql.download -- "$sql_url" &&
    mv -- input.sql.download input.sql &&
    head -n 3 input.sql
```

В Windows PowerShell на учебной машине выполните `Set-Location -LiteralPath 'C:\Practice07'`, затем отдельно `$SqlUrl = Read-Host 'Адрес ссылки на SQL'`. Вставьте адрес и нажмите Enter. Затем выполните следующий блок.

```
$Download = @{
    Uri = $SqlUrl
    UseBasicParsing = $true
    OutFile = 'input.sql.download'
    ErrorAction = 'Stop'
}
try {
    Invoke-WebRequest @Download
    Move-Item input.sql.download input.sql -Force
    Get-Content input.sql -Encoding UTF8 -TotalCount 3
} catch {
    Write-Warning $_.Exception.Message
}
```

7

## Если учебная машина может скачать файл сама · продолжение

Если скачивание выдало ошибку, остановитесь и воспользуйтесь следующим способом. Файл с расширением `.download` мог записаться не полностью.

8

## Если файл скачан на ваш компьютер

Откройте скачанный `variant-NN.sql` текстовым редактором на своём компьютере. Проверьте номер в первой строке. Если паспорт среды указывает общую папку для передачи, скопируйте туда файл через Проводник, откройте эту папку на учебной машине и скопируйте файл в рабочую папку под именем `input.sql`.

Если доступно копирование текста в учебный сеанс, выделите весь SQL в редакторе (Ctrl+A, Ctrl+C). На учебной Linux-машине в рабочей папке выполните `nano input.sql`. В открытом редакторе вставьте текст через меню терминала «Вставить» и сохраните Ctrl+O, Enter, Ctrl+X. В Windows выполните `notepad.exe C:\Practice07\input.sql` на учебной машине, вставьте текст в Блокнот и сохраните файл в UTF-8 под точным именем `input.sql`. Если доступна только консоль без вставки, запросите у преподавателя выданный способ переноса и путь назначения.

9

## Проверьте файл перед подключением

В рабочей папке на учебной машине проверьте имя, размер и первые строки файла. Найдите в нём проверочную вставку.

```
ls -l input.sql
head -n 3 input.sql
grep -n -F 'INSERT INTO access_probe VALUES (1,10);' input.sql
```

```
Get-Item -LiteralPath input.sql
Get-Content -LiteralPath input.sql -Encoding UTF8 -TotalCount 3
$Marker = 'INSERT INTO access_probe VALUES (1,10);'
Select-String -Path input.sql -Encoding UTF8 -SimpleMatch $Marker
```

В первой строке должен быть ваш номер, а поиск должен найти строку `INSERT INTO access_probe VALUES (1,10);`. Если команда сообщает об отсутствии файла, сначала проверьте рабочую папку и расширение `.sql`.

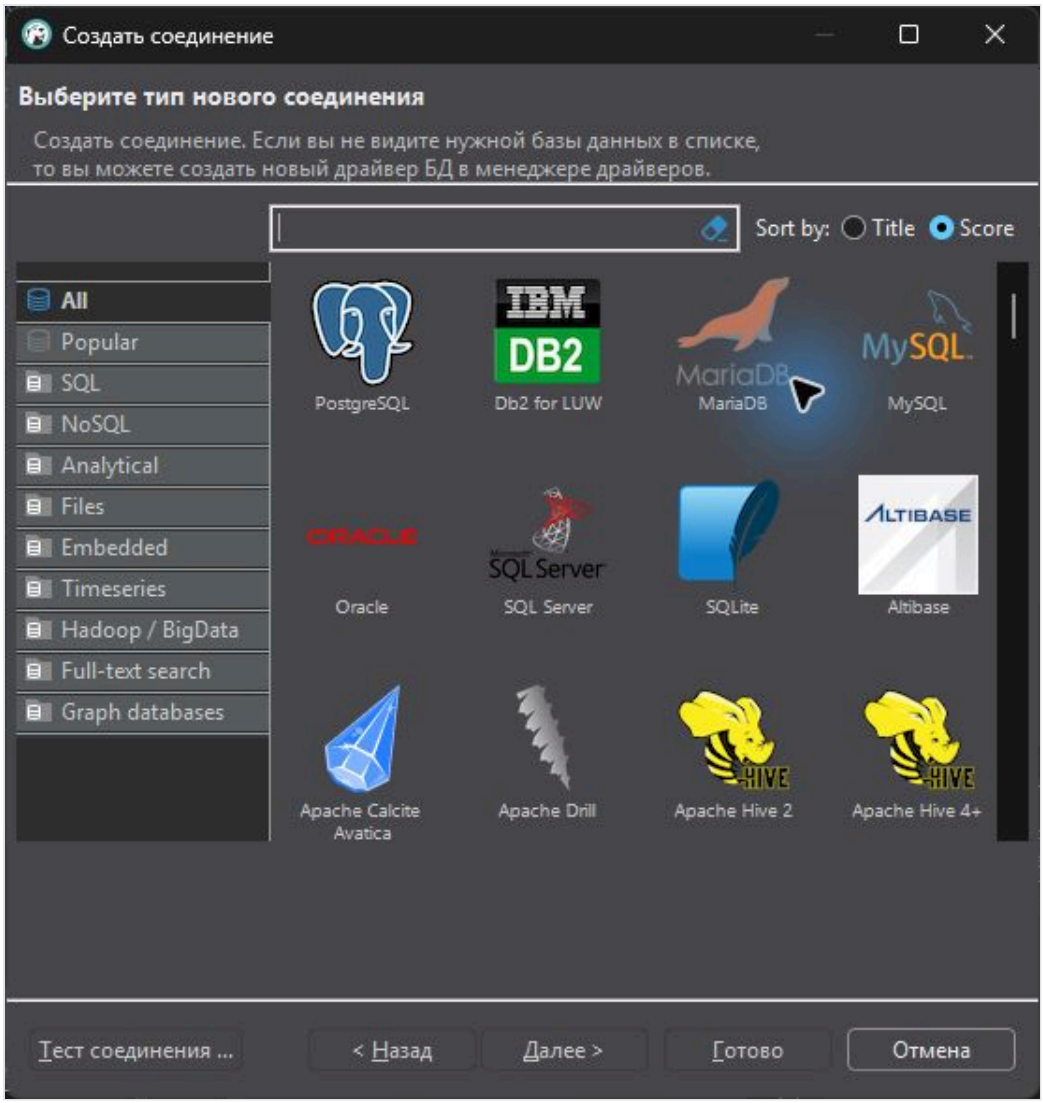
Вернитесь в свой маршрут и откройте шаг входа начальным администратором. Перед загрузкой замените имена базы, таблиц и файлов по таблице подстановок. В результате первого контрольного запроса по каждой предметной таблице ожидаются 4 строки, сумма `id` 10 и сумма `amount` 100.

# Подключение в DBeaver: разбор на образце

10

## Выберите свой драйвер

На снимках показан PostgreSQL и учебная база pm07_demo. Сначала «База данных» → «Новое соединение» либо Ctrl+Shift+N. Выберите PostgreSQL, MySQL, MariaDB или SQL Server **по своей карточке**, затем «Далее». Драйвер PostgreSQL не подходит для MySQL, даже если имя базы одинаковое.

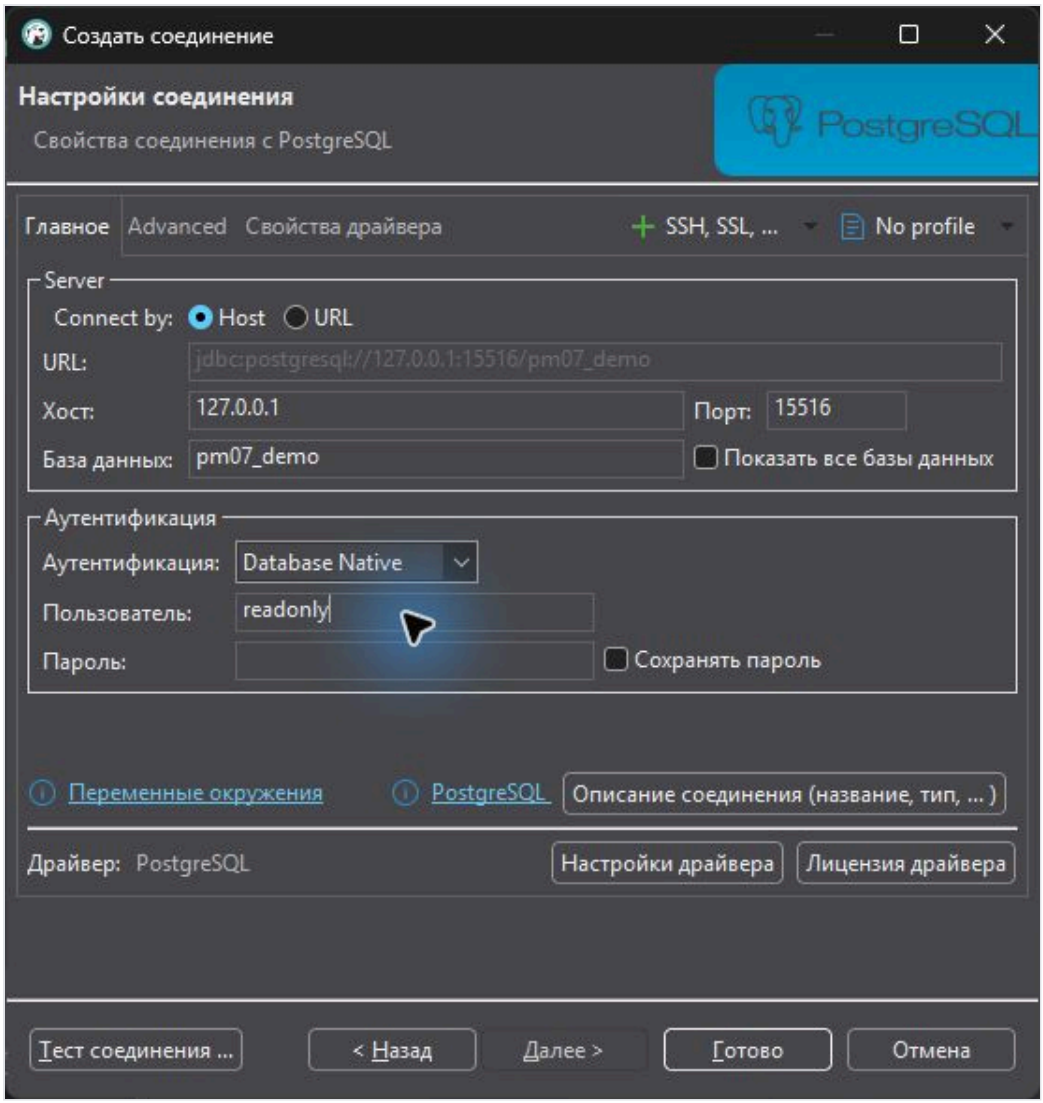



Окно выбора драйвера в DBeaver

11

## Перепишите параметры из паспорта

В примере сервер доступен с хоста по 127.0.0.1:15516. У себя перепишите адрес и порт именно клиентского подключения, имя своей базы и readonly. Отключите «Сохранять пароль», затем нажмите «Тест соединения». Названия полей могут быть на русском или английском языке.

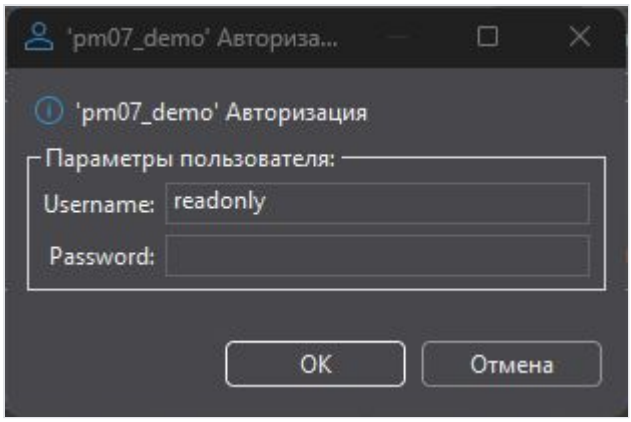



Параметры PostgreSQL без сохранения пароля

12

## Введите пароль в отдельном окне

Если сохранение отключено, поле Password основной формы недоступно. Введите пароль readonly в появившемся окне авторизации. При запросе Download driver используйте заранее подготовленный преподавателем клиент. Успешный тест должен сообщить об установленном соединении; затем «Готово». При первом открытии консоли пароль может запрашиваться снова: это не ошибка входа.




Отдельный запрос пароля выбранной роли

13

## Откройте консоль и выполните запрос

Выделите своё соединение слева. В меню «Редактор SQL» выберите «Open SQL console», либо нажмите Ctrl+Alt+Enter. Вставьте контрольный SELECT своего маршрута; Ctrl+Enter выполняет текущий запрос. В примере три SELECT объединены UNION ALL, чтобы одновременно видеть все предметные таблицы.

```
SELECT current_user AS role_name,'devices' AS table_name,
  COUNT(*) AS row_count,SUM(id) AS id_sum,
  SUM(amount) AS amount_sum FROM devices
UNION ALL
SELECT current_user,'locations',COUNT(*),
  SUM(id),SUM(amount) FROM locations
UNION ALL
SELECT current_user,'checks',COUNT(*),
  SUM(id),SUM(amount) FROM checks;
```

Действительно выполненный контрольный запрос

14

## Сопоставьте результат с CLI

Внизу появляются строки результата, а не только текст запроса. В образце у каждой таблицы 4 строки, сумма id 10, сумма amount 100; роль readonly. У себя должны совпасть имена и значения своего SQL-файла. Пустая таблица результатов, запрос к чужой базе или ошибка входа не подтверждают выполнение.

| | A-Z role_nan | A-Z table_na | 123 row_cou | 123 id_s | 123 amount_sum |
|---|---|---|---|---|---|
| 1 | readonly | devices | 4 | 10 | 100 |
| 2 | readonly | locations | 4 | 10 | 100 |
| 3 | readonly | checks | 4 | 10 | 100 |

Результат трёх контрольных запросов

ПМ.07 / Соадминистрирование

# Кто управляет чем

1

| Учётная запись | Разрешено | Не входит в её работу |
|---|---|---|
| Начальный администратор сервера | Создать базу, учётные записи и выдать права | Использовать его вместо всех проверяемых ролей |
| admin | Создавать и удалять учебные таблицы, работать с данными своей базы, делать её копию | Создавать серверные учётные записи на основании только прав на одну базу |
| app_user | SELECT, INSERT, UPDATE и DELETE в выданных таблицах | Создавать таблицы и серверные учётные записи |
| readonly | SELECT | Изменять данные и структуру |

ПМ.07 / Соадминистрирование

# Как выполнять проверку

2

Используйте свой маршрут: там приведён синтаксис именно своей СУБД. Каждая роль проверяется новым подключением. Начальный администратор не используется для действий, которые должен выполнить `app_user` или `readonly`.

Сначала выполните разрешённый SELECT, затем предусмотренный запрет. Ошибка авторизации при входе означает, что роль ещё не проверена. Ошибка SQL-синтаксиса тоже не является доказательством запрета. В журнал переписывается название отклонённой операции и понятная причина отказа.

ПМ.07 / Соадминистрирование

# Что сравнивать

3

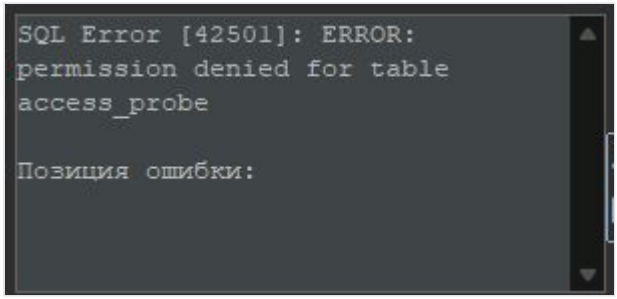
Видимость служебных баз и схем у разных СУБД различается. Сам список имён не доказывает возможность изменения данных. Проверяются конкретные права и конкретные операции в своей учебной базе. В выводе отчёта не заменяйте это утверждением «ничего лишнего не видно».

# Пример ожидаемого отказа

4

## Проверяйте причину сообщения

После успешного SELECT в том же соединении readonly выполните отдельно `INSERT INTO access_probe VALUES (2, 20);`. В PostgreSQL ожидается SQLSTATE 42501 и `permission denied for table access_probe`, как на реальном снимке. В других СУБД формулировка и код отличаются; сравнивайте причину, указанную в своём маршруте. Соединение уже установлено, поэтому этот отказ подтверждает запрет записи. Затем отдельно проверьте DELETE и запреты app_user; один снимок INSERT их не заменяет.




PostgreSQL отклонил запись от readonly

ПМ.07 / Соадминистрирование

# До копирования

1

Сохраните контрольные значения своей базы: по каждой выданной таблице `COUNT(*)`, `SUM(id)` и `SUM(amount)`. В исходном наборе каждая из трёх предметных таблиц содержит 4 строки, сумму id 10 и сумму amount 100. Таблица `access_probe` проверяется отдельно. Если собственная операция меняла данные, зафиксируйте новые значения перед резервированием.

ПМ.07 / Соадминистрирование

# Какую базу восстанавливать

2

Восстановление первого бэкапа выполняется в новую `pm07_vNN_restore`. Исходную `pm07_vNN` не удаляют и не переводят в SINGLE_USER ради этой проверки. Точное имя файла берётся из только что выполненной команды; звёздочка в имени не выбирает нужную копию за студента.

ПМ.07 / Соадминистрирование

# Как понять результат

3

Последовательность команд находится в маршруте своей СУБД. После восстановления сравните все таблицы и контрольные значения. Совпадение числа строк при разных суммах или отсутствующей таблице не является совпадением данных.

`pg_restore --list`, `RESTORE VERIFYONLY`, ненулевой размер файла и успешный код команды дают частичные сведения. Итоговая проверка этой практики включает восстановление и запросы к новой базе. SHA-256 подтверждает неизменность конкретного файла между двумя измерениями.

ПМ.07 / Соадминистрирование

# Если значения различаются

4

Сначала сопоставьте имена исходной/проверочной базы, точное имя и время создания файла, список включённых таблиц и сообщения команды восстановления. Остановитесь на первой причине. Различие не доказывает автоматически, что «копии делаются слишком редко».

# Разбор восстановления PostgreSQL на образце

5

## Сначала зафиксируйте данные

Реальный запрос CLI к pm07_demo вернул по 4 строки, сумму id 10 и сумму amount 100. В графическом клиенте получены те же значения. В SQL своего варианта имена таблиц заменяются до запуска; сравнивайте все три, а не одну удобную таблицу.

```
 table_name | row_count | id_sum | amount_sum
------------+-----------+--------+------------
 devices    |         4 |     10 |     100.00
 locations  |         4 |     10 |     100.00
 checks     |         4 |     10 |     100.00
(3 rows)
```

Контрольные значения исходной базы из psql

6

## Создайте архив и запишите точный файл

Ниже показан результат действительного pg_dump с форматом -Fc, затем размер, SHA-256 и фрагмент списка архива. Здесь файл находится в /tmp/manual.dump; у себя используйте каталог своего маршрута. Строки с «+» показывают выполняемые команды оболочки и не входят в текст, который нужно копировать. Код 0 означает завершение команды, но ещё не проверку восстановления.

```
+ pg_dump -h 127.0.0.1 -U admin -d pm07_demo -Fc -f /tmp/manual.dump
+ printf exit=%s\n 0
exit=0
+ ls -lh /tmp/manual.dump
-rw-r--r-- 1 root root 6.4K Nov  4 21:18 /tmp/manual.dump
+ sha256sum /tmp/manual.dump
bc2336ffb8b700da224b0b19e88cff334bf7a119c017bfb1ae0a2351161c7df5  /tmp/manual.dump
+ pg_restore --list /tmp/manual.dump
+ tail -n 6
3367; 0 16795 TABLE DATA public locations admin
3222; 2606 16809 CONSTRAINT public access_probe access_probe_pkey admin
3220; 2606 16804 CONSTRAINT public checks checks_pkey admin
3216; 2606 16794 CONSTRAINT public devices devices_pkey admin
3218; 2606 16799 CONSTRAINT public locations locations_pkey admin
2049; 826 16810 DEFAULT ACL public DEFAULT PRIVILEGES FOR TABLES admin
```

Созданный архив PostgreSQL и его SHA-256

7

## Сверьте отдельную восстановленную базу

Архив восстановлен в заранее выделенную пустую pm07_demo_shot_restore. Это отдельное имя проверочной базы; исходная pm07_demo сохранена. Запрос выполняется уже к назначению и возвращает те же три результата 4/10/100. У себя используйте имя _restore из таблицы подстановок. Синтаксис восстановления для MySQL, MariaDB и SQL Server берётся из своего маршрута, а не из этого примера PostgreSQL.

```
    current_database    | table_name | row_count | id_sum | amount_sum
------------------------+------------+-----------+--------+------------
 pm07_demo_shot_restore | devices    |         4 |     10 |     100.00
 pm07_demo_shot_restore | locations  |         4 |     10 |     100.00
 pm07_demo_shot_restore | checks     |         4 |     10 |     100.00
(3 rows)
```

Три таблицы отдельной восстановленной базы

ПМ.07 / Соадминистрирование

# Что автоматизировать

1

Ручная резервная копия обязательна для всех, а операция по расписанию определяется карточкой варианта. Это может быть бэкап, отчёт, проверка места, обслуживание или другая указанная операция. Не сдавайте ежедневный бэкап вместо отчёта по своему варианту.

ПМ.07 / Соадминистрирование

# Как устроен скрипт

2

В своём маршруте возьмите каркас Bash или PowerShell, а из карточки — содержимое `run_task` / `Invoke-Task`. Вставка заменяет указанную функцию целиком; параметры и служебные функции берутся из того же маршрута. Названия переменных в карточке и каркасе должны совпадать.

Скрипт фиксирует начало, результат операции, конец и код возврата. Пароль хранится в профиле клиента вне скрипта. Начальный каталог, пользователь ОС и доступ к файлам должны совпадать при ручном и запланированном запуске. Профиль одного пользователя Windows нельзя считать автоматически доступным SYSTEM.

# Сначала вручную

3

Подготовьте вход для проверки из своей карточки и запустите скрипт. Сверьте код 0, свежую строку END и результат из карточки. Например, для сжатия архивов нужен хотя бы один исходный файл. Сохраните доказательства успешного запуска до отрицательного опыта. Для проверки отказа выберите условие по следующей таблице и используйте отдельную копию.

| **Операция** | **Как вызвать отказ** |
|---|---|
| Читает или меняет БД | В тестовой конфигурации укажите `pm07_missing_test` вместо имени рабочей базы. |
| Проверяет место, сжимает архивы, запускает службу или читает журнал, варианты 02, 05, 06, 17 | Выполните опыт из своей карточки. Эти операции не проверяют существование базы. |

4

## Копия для опыта на Linux

В рабочей папке выполните `cp -- run.sh run-test.sh` и `cp -- config.env config-test.env`. В `nano run-test.sh` найдите строку `source "$WORK/config.env"` и замените в ней имя файла на `config-test.env`. Измените условие из таблицы в `config-test.env` либо в тестовой функции, если карточка указывает на функцию. Выполните `bash ./run-test.sh`, сразу после него `echo $?`, затем `tail -n 8 logs/run.log`. Ожидаются ненулевой код и свежая строка ERROR. Для варианта 05 используйте отдельную папку по карточке вместо этой пары файлов.

5

### Копия для опыта на Windows

В PowerShell в рабочей папке выполните `Copy-Item -LiteralPath run.ps1 -Destination run-test.ps1` и `Copy-Item -LiteralPath config.ps1 -Destination config-test.ps1`. Откройте `notepad.exe run-test.ps1`. В строке `. (Join-Path $Work 'config.ps1')` замените только имя файла на `config-test.ps1`. Откройте тестовую конфигурацию, поставьте в `$Database` значение `pm07_missing_test` и сохраните. Выполните `powershell.exe -NoProfile -File .\run-test.ps1`, сразу после него `$LASTEXITCODE`, затем `Get-Content -LiteralPath .\logs\run.log -Tail 8`.

Сопоставьте причину ошибки с изменённым условием. Запишите код и строку ERROR в раздел отчёта об отказе. Для следующего рабочего опыта запускайте исходный `run.sh` или `run.ps1` с исходным конфигом.

ПМ.07 / Соадминистрирование

# Индивидуальное дополнение

6

Выберите одно содержательное дополнение из списка областей. Базовые ERROR/INFO, отсутствие пароля в скрипте и проверка кода обязательны для всех; их наличие само по себе не является новым дополнением. Покажите, что именно добавлено относительно взятого образца и каким отдельным опытом это проверено.

7

## Образец на Bash: измерьте длительность

В своей рабочей папке сначала скопируйте `run.sh` в `run-timer.sh`. Откройте копию командой `nano run-timer.sh`. Внизу найдите отдельную строку вызова `run_task` между записями START и END. Замените только эту строку следующим блоком. Определение `run_task() { ... }` в середине файла сохраняется.

```
started_ms="$(date +%s%3N)"
run_task
finished_ms="$(date +%s%3N)"
log INFO "elapsed_ms=$((finished_ms - started_ms))"
```

Перед запуском проверьте вход. Если операция меняет данные, используйте восстановленную проверочную базу в тестовой конфигурации. Если операция удаляет или переносит файлы, подготовьте новую копию входных файлов. Выполните `bash ./run-timer.sh`, `echo $?` и `tail -n 8 logs/run.log`. При успехе между START и END появится строка `elapsed_ms=...` с числом миллисекунд. Для контрольного опыта можно добавить `sleep 1` сразу после первой строки блока. При успешной операции замер должен быть не меньше 1000 мс. Уберите паузу перед переносом блока в рабочий `run.sh`.

8

## Образец на PowerShell: измерьте длительность

Выполните `Copy-Item -LiteralPath run.ps1 -Destination run-timer.ps1`, затем `notepad.exe run-timer.ps1`. В блоке `try` найдите отдельную строку `Invoke-Task` между START и END. Вместо неё вставьте следующий блок.

```
$Timer = [System.Diagnostics.Stopwatch]::StartNew()
Invoke-Task
$Timer.Stop()
Write-Log INFO "elapsed_ms=$($Timer.ElapsedMilliseconds)"
```

Для операции, меняющей данные или файлы, сначала выберите проверочную базу либо подготовьте копию файлов. Запустите `powershell.exe -NoProfile -File .\run-timer.ps1`, проверьте `$LASTEXITCODE` и последние строки `logs\run.log`. Для контрольного опыта в копии можно добавить `Start-Sleep -Seconds 1` после первой строки блока. После проверки уберите паузу и перенесите проверенный блок в рабочий `run.ps1`.

9

### Перенесите приём на свой вариант

Если ваша операция уже пишет длительность, выберите другой пункт из списка дополнений. В отчёте укажите имя файла, место правки и фактический результат отдельного опыта. Для операции, которая меняет данные или удаляет файлы, проверьте дополнение на отдельной копии входных данных. Перед расписанием выполните итоговый рабочий скрипт вручную.

ПМ.07 / Соадминистрирование

## Затем по расписанию

10

Используйте ту же учётную запись ОС, что прошла ручную проверку. На Windows задайте ближайший однократный триггер. На Linux добавьте временную строку cron из своего маршрута, дождитесь первого срабатывания и затем замените её постоянной. Сопоставьте время планировщика, новую запись в журнале, код и фактический результат. Если периодичность в карточке не указана, выберите её и объясните в отчёте.

# Подготовьте репозиторий и сдайте отчёт

11

## Отберите файлы для преподавателя

На своём компьютере создайте отдельную папку для передачи. Положите туда исходный `input.sql`, SQL настройки прав с учебными обозначениями вместо паролей, итоговый `run.sh` или `run.ps1`, пример конфигурации с пустыми секретами и `README.md`. Добавьте требуемые для повторения несекретные входные данные, протокол проверок и экспорт расписания, если они используются. Журнал, резервную копию и доказательства сохраните для сдачи по условиям карточки. Пароли, токены, парольный профиль из папки `auth` и личные данные исключите из файлов для репозитория. Входные скрипты откройте в редакторе и проверьте их содержимое до загрузки.

Если работа находится на учебной машине, откройте выданную общую папку на учебной машине и скопируйте в неё отобранные файлы. Затем откройте ту же папку на своём компьютере и перенесите файлы в папку передачи. При работе только с буфером обмена откройте каждый текстовый файл на учебной машине, скопируйте его целиком и сохраните на своём компьютере под тем же именем. Для резервной копии и снимков используйте выданный канал передачи файлов. Откройте перенесённые скрипт, SQL и журнал на своём компьютере перед загрузкой.

12

## Отберите файлы для преподавателя · продолжение

Создайте `README.md` в текстовом редакторе. Укажите номер и область, ОС и СУБД, назначение файлов, какой конфиг создать из примера, команду ручного запуска, периодичность, расположение журнала и ожидаемый результат. Сохраните файл в UTF-8 с расширением `.md`.

13

## Загрузите файлы через браузер

1. Войдите в GitHub. В меню + справа вверху нажмите `New repository`. В поле имени введите, например, `pm07-v05`, подставив свой номер. Выберите видимость по указанию преподавателя, включите создание README и нажмите `Create repository`.
2. На странице своего репозитория откройте `Add file → Upload files` и загрузите SQL, скрипт и остальные подготовленные файлы. Сохраните изменения кнопкой `Commit changes`. Для `README.md` откройте уже созданный файл, нажмите значок карандаша, вставьте свой текст и сохраните изменения. В личном учебном репозитории сохраните работу в основной ветке, чтобы файлы открывались по ссылке на главную страницу.
3. Откройте главную страницу репозитория и проверьте, что видны `README.md`, ваш скрипт и SQL нужного варианта. Откройте по одному файлу каждого типа. Скопируйте адрес главной страницы из строки браузера, например `https://github.com/USERNAME/pm07-v05`. В отчёте замените пример своей ссылкой.

14

## Проверьте доступ и передайте результат

Публичный репозиторий откройте по скопированной ссылке в приватном окне браузера. Для приватного репозитория предоставьте доступ учётной записи преподавателя по выданной инструкции. Если используется личный GitHub, в своём репозитории откройте `Settings → Collaborators → Add people`, найдите точный логин преподавателя и проверьте, что приглашение принято. До принятия приглашения проверяющий может получить ошибку 404. В учебной организации используйте выданный способ доступа.

Откройте «Отчёт о выполнении проекта.docx» из ZIP своей группы. В разделе «Итоговая проверка и сдача» вставьте адрес в поле ссылки на репозиторий. В Word можно выделить адрес и нажать Ctrl+K, чтобы сделать его ссылкой. Сохраните DOCX и откройте ссылку из сохранённого файла.

15

### Прикрепите комплект в личном кабинете

Сверьте срок и состав комплекта по «Пояснительной записке к комплекту отчётной документации» из ZIP своей группы. Перенесите итоги и доказательства в презентацию «Приложение 1 к индивидуальному заданию». В PowerPoint для Windows выберите «Файл → Экспорт → Создать документ PDF/XPS» и откройте сохранённый PDF. Проверьте, что в нём читаются результаты.

В личном кабинете откройте задание по учебной практике ПМ.07 своей группы. В поле «Ответ» прикрепите подписанное индивидуальное задание в PDF, презентацию в PDF, «Отчёт о выполнении проекта.docx» и аттестационный лист в DOCX. В аттестационном листе заполните только первый абзац по пояснительной записке. Если в вашей выдаче указаны дополнительные формы, используйте их по той же записке. После отправки убедитесь, что в ответе видны прикреплённые файлы и у каждого правильное расширение.

ПМ.07 / Соадминистрирование

# Как читать фактические результаты

16

### Успех ручной операции

В примере варианта 07 выполнен bash ./run.sh: код 0, новая START, путь созданного custom-архива и END result=OK. Журнал хранит историю, поэтому более ранний ERROR остаётся видимым выше. Сопоставляйте время **своего нового запуска**, точное имя базы и существование результата. У другого варианта результат может быть CSV, отчётом или состоянием службы, а не архивом.

```
+ bash ./run.sh
+ printf exit=%s\n 0
exit=0
+ tail -n 5 logs/run.log
2026-11-04T19:29:07+00:00 [INFO] START db=pm07_missing_test
2026-11-04T19:29:07+00:00 [ERROR] exit=1
2026-11-04T21:19:31+00:00 [INFO] START db=pm07_v07
2026-11-04T21:19:31+00:00 [INFO] custom_archive=/qa/pm07_v07/backups/pm07_v07_20261104_211931.dump
2026-11-04T21:19:31+00:00 [INFO] END result=OK
+ ls -lh backups
+ tail -n 3
-rw------- 1 root root 1.3K Nov  4 19:29 pm07_v07_20261104_192907.dump.toc.txt
-rw------- 1 root root 6.5K Nov  4 21:19 pm07_v07_20261104_211931.dump
-rw------- 1 root root 1.3K Nov  4 21:19 pm07_v07_20261104_211931.dump.toc.txt
```

Ручной запуск варианта 07 и настоящий журнал

17

## Отказ проверочной копии

В отдельном config-test.env указана pm07_missing_test. Запущен run-test.sh. pg_dump сообщает об отсутствующей базе, код равен 1, последняя запись журнала — ERROR exit=1. Для варианта 02, 05, 06 или 17 выберите отказ из своей карточки.

```
+ bash ./run-test.sh
pg_dump: error: connection to server at "127.0.0.1", port 5432 failed: FATAL:  database "pm07_missing_test" d
oes not exist
+ printf exit=%s\n 1
exit=1
+ tail -n 5 logs/run.log
2026-11-04T21:19:31+00:00 [INFO] START db=pm07_v07
2026-11-04T21:19:31+00:00 [INFO] custom_archive=/qa/pm07_v07/backups/pm07_v07_20261104_211931.dump
2026-11-04T21:19:31+00:00 [INFO] END result=OK
2026-11-04T21:19:55+00:00 [INFO] START db=pm07_missing_test
2026-11-04T21:19:55+00:00 [ERROR] exit=1
```

Контролируемый отказ отдельной копии скрипта

18

## Действительный запуск cron

В реальном примере варианта 06 crontab -l содержит один запуск по чётным минутам. В системном журнале есть CMD и END в 21:22 UTC; «+00:00» — часовой пояс записи. При сравнении с локальным временем учитывайте пояс своего паспорта. Этот снимок сделан после наступления времени, без нажатия ручного запуска.

```
+ crontab -l
SHELL=/bin/bash
*/2 * * * * /bin/bash "$HOME/pm07/run.sh" >> "$HOME/pm07/logs/cron.log" 2>&1
+ date -Is
2026-11-04T21:22:33+00:00
+ + tailgrep -n 4
 CMD\|END /var/log/syslog
2026-11-04T21:20:01.809647+00:00 50c78def830b CRON[1598]: (root) CMD ([1599] /bin/bash "$HO
ME/pm07/run.sh" >> "$HOME/pm07/logs/cron.log" 2>&1)
2026-11-04T21:20:01.847700+00:00 50c78def830b CRON[1598]: (root) END ([1599] /bin/bash "$HO
ME/pm07/run.sh" >> "$HOME/pm07/logs/cron.log" 2>&1)
2026-11-04T21:22:01.847157+00:00 50c78def830b CRON[1630]: (root) CMD ([1631] /bin/bash "$HO
ME/pm07/run.sh" >> "$HOME/pm07/logs/cron.log" 2>&1)
2026-11-04T21:22:01.888946+00:00 50c78def830b CRON[1630]: (root) END ([1631] /bin/bash "$HO
ME/pm07/run.sh" >> "$HOME/pm07/logs/cron.log" 2>&1)
```

Запись crontab и состоявшийся триггер

19

## Подтвердите действие после триггера

За те же 21:22 UTC в журнале скрипта появились START, service already active и END result=OK. Проверка состояния службы вернула код 0, запрос к products — 4/10/100. Если служба уже работает, её повторный запуск не требуется. Отдельный обязательный опыт варианта 06 включает также запуск остановленной учебной службы; одна эта картинка его не заменяет. Парольный профиль в снимки не входит.

```
+ grep 2026-11-04T21:22: /root/pm07/logs/run.log
2026-11-04T21:22:01+00:00 [INFO] START db=pm07_v06
2026-11-04T21:22:01+00:00 [INFO] service already active
2026-11-04T21:22:01+00:00 [INFO] END result=OK
+ /usr/sbin/service mariadb status
+ printf service_status_exit=%s\n 0
service_status_exit=0
+ mariadb -uroot -e SELECT COUNT(*),SUM(id),SUM(amount) FROM pm07_v06.products;
+----------+---------+-------------+
| COUNT(*) | SUM(id) | SUM(amount) |
+----------+---------+-------------+
|        4 |      10 |      100.00 |
+----------+---------+-------------+
+ date -Is
2026-11-04T21:24:02+00:00
```

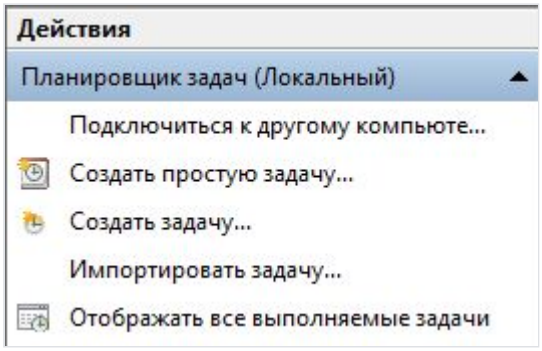
Журнал того же запуска, состояние службы и данные

# Расписание в Windows

20

## Откройте создание задачи

Запустите «Планировщик задач» через Пуск. На начальном экране справа выберите «Создать задачу...». Это вход в форму с вкладками «Общие», «Триггеры» и «Действия». Заполняйте их по таблице своего Windows-маршрута. «Создать простую задачу...» открывает другой мастер, поэтому названия следующих шагов в нём отличаются.




Начальная панель действий Планировщика задач

21

## Сопоставьте три поля действия

Программа содержит только путь к powershell.exe; аргументы — -NoProfile -File и полный путь к своему run.ps1; рабочая папка — каталог этого скрипта без имени файла. На вкладке «Общие» оставьте свою проверенную учётную запись Windows и запуск для вошедшего пользователя. На вкладке «Триггеры» создайте один ближайший однократный запуск. Точное заполнение приведено в своём маршруте; значения из Linux-cron сюда не переносятся.

22

### Проверьте наступивший запуск

После назначенного времени откройте «Библиотека планировщика задач» слева, выберите своё имя PM07_VNN и обновите список клавишей F5. Сопоставьте «Время прошлого запуска» и «Результат последнего запуска» с новой записью START/END своего журнала и фактическим файлом или результатом. При необходимости запросите те же данные из PowerShell:

```
Get-ScheduledTaskInfo -TaskName 'PM07_VNN' |
    Select-Object LastRunTime,LastTaskResult,NextRunTime
```

Замените NN номером своей задачи. LastTaskResult=0 означает успешное завершение процесса; дополнительно проверьте результат операции. Общая сводка на начальном экране не заменяет свойства своей задачи, а ручная команда запуска не подтверждает наступление триггера. После опыта измените существующий триггер на постоянную периодичность своей карточки и экспортируйте XML через правую панель выбранной задачи.

Маршрут / MySQL 8 · Linux

# Прежде чем копировать команды

1

### Работайте только в этом маршруте

ОС и СУБД должны совпадать с карточкой варианта. Здесь все команды ОС выполняются в: **терминал учебной Linux-машины**. SQL выполняется отдельно в клиенте **mysql**. Адрес клиента на хосте и адрес внутри VM могут различаться: перепишите оба из паспорта учебной среды. Настройка сервера и канал доступа уже подготовлены; установка из другого маршрута не выполняется.

2

## Заполните подстановки

| В образце | В своём варианте |
|---|---|
| pm07_demo | pm07_vNN — имя исходной базы |
| pm07_demo_restore | pm07_vNN_restore — отдельная проверочная база |
| devices, locations, checks | Три имени таблиц из своей карточки |
| input.sql | Скачанный SQL своего варианта, сохранённый под этим рабочим именем |
| ADMIN_PASSWORD, APP_PASSWORD, READ_PASSWORD | Собственные учебные пароли, подставляемые до выполнения SQL |

В SQL сохраняйте обозначение имени таблицы: обратные кавычки вокруг имени для MySQL/MariaDB. Заменяется имя внутри них. Это важно, например, для исходной таблицы groups.

3

## Заполните подстановки · продолжение

Во всех блоках SQL сначала замените длинное имя pm07_demo_restore, затем pm07_demo. Сохраняйте каждый изменённый блок в текстовом редакторе до запуска. Пароль в SQL-строке заключён в одинарные кавычки; внутреннюю одинарную кавычку удваивают. При подготовке профиля клиента учитывайте его собственное экранирование. Пароли не помещаются в снимки отчёта.

Маршрут / MySQL 8 · Linux

# 1. Подключение и исходные данные

4

## Подготовьте рабочую папку

```
mkdir -p "$HOME/pm07"
cd "$HOME/pm07"
pwd
```

Сохраните в этой папке SQL своей карточки под именем input.sql. Если файл скачан на другой компьютер, выполните перенос и проверку файла. Разбор использует pm07_demo; в своих командах укажите имя базы по номеру варианта.

5

## Войдите начальным администратором

Используйте выданную команду входа в mysql. При парольной TCP-аутентификации пример в терминале: `mysql -h 127.0.0.1 -P 3306 -u root -p`. Если в паспорте указан локальный socket-вход, используйте именно его. Создание базы и всех учётных записей ниже выполняется начальным администратором сервера, а не пользователем admin одной базы.

В отдельной учебной VM вход для выданных клиентов заранее ограничен подготовленным каналом. В примере имена MySQL-учётных записей имеют host-часть %, чтобы одна учебная роль могла пройти локальную и выданную клиентскую проверку. Эта часть — условие сопоставления клиента, а не команда настройки сетевого доступа.

```
SELECT VERSION(), CURRENT_USER(), DATABASE();
CREATE DATABASE pm07_demo CHARACTER SET utf8mb4;
CREATE DATABASE pm07_demo_restore CHARACTER SET utf8mb4;
CREATE USER 'admin'@'%' IDENTIFIED BY 'ADMIN_PASSWORD';
CREATE USER 'app_user'@'%' IDENTIFIED BY 'APP_PASSWORD';
CREATE USER 'readonly'@'%' IDENTIFIED BY 'READ_PASSWORD';
GRANT ALL PRIVILEGES ON pm07_demo.* TO 'admin'@'%';
GRANT ALL PRIVILEGES ON pm07_demo_restore.* TO 'admin'@'%';
GRANT SELECT,INSERT,UPDATE,DELETE ON pm07_demo.* TO 'app_user'@'%';
GRANT SELECT ON pm07_demo.* TO 'readonly'@'%';
SHOW GRANTS FOR 'admin'@'%';
SHOW GRANTS FOR 'app_user'@'%';
SHOW GRANTS FOR 'readonly'@'%';
```

6

## Загрузите выданный SQL от admin

Завершите соединение командой quit и войдите заново как admin в исходную базу. Полный путь к клиенту Windows берётся из паспорта. В консоли SQL выберите свою базу через USE, затем выполните SOURCE с полным путём к input.sql. На Windows путь записывается C:/Practice07/input.sql; на Linux — значение pwd и /input.sql. Это команда клиента SQL, а не строка PowerShell.

GRANT на pm07_demo.* не выдаёт глобальную привилегию CREATE USER. В этом маршруте серверные учётные записи уже созданы начальным администратором; проверка admin относится к своей базе. Дополнительный FLUSH PRIVILEGES после CREATE USER и GRANT не требуется.

7

## Проверьте исходный набор и графический клиент

Подключитесь к исходной базе как admin и выполните для каждой своей предметной таблицы (в примере devices):

```
SELECT COUNT(*) AS row_count, SUM(id) AS id_sum, SUM(amount) AS amount_sum FROM `devices`;
```

Ожидаются 4, 10, 100. Сохраните три результата с именами таблиц. access_probe содержит id=1 и value=10 и в эти три предметных результата не входит.

8

## Проверьте исходный набор и графический клиент · продолжение

На хосте откройте выданный графический клиент. В DBeaver: «База данных» (Database) → «Новое соединение» (New Database Connection) → драйвер своей СУБД → «Далее» (Next); введите адрес и порт проброса **из паспорта хоста**, имя исходной базы, readonly и его пароль → «Тест соединения» (Test Connection) → «Готово» (Finish). Если отключено «Сохранять пароль», введите пароль в появившемся отдельном окне авторизации; поле пароля в основной форме при этом недоступно. Если клиент просит драйвер, используйте выданный заранее настроенный клиент; загрузка неизвестного драйвера не заменяет подготовку среды. Откройте «Редактор SQL» (SQL Editor) → «Open SQL console» (Ctrl+Alt+Enter) и выполните SELECT COUNT(*) для одной своей таблицы. Пошаговый разбор со снимками находится в общем шаге подключения. В SSMS аналогично выбираются выданный сервер и SQL Server Authentication с readonly, затем New Query и своя база. Результат должен совпасть с CLI. 127.0.0.1 внутри VM и на хосте означает разные машины; порт из гостевого примера не переносится в хост автоматически.

# 2. Три отдельных подключения и права

9

## Позитивные опыты

Для каждого опыта создайте новое соединение и проверьте имя текущей роли. Под admin выполните создание и удаление только учебной check_admin. Под app_user измените отдельную строку access_probe и затем верните её в исходное состояние:

```
CREATE TABLE check_admin (id int);
DROP TABLE check_admin;
```

```
INSERT INTO access_probe(id,value) VALUES(9001,20);
UPDATE access_probe SET value=30 WHERE id=9001;
SELECT * FROM access_probe WHERE id=9001;
DELETE FROM access_probe WHERE id=9001;
```

Первый блок — только admin; второй — только app_user. Затем readonly читает SELECT * FROM access_probe. После опыта остаётся ровно исходная строка id=1, value=10.

10

### Негативные опыты

Под readonly отдельно проверьте INSERT в access_probe и DELETE строки id=1. Под app_user отдельно проверьте CREATE TABLE forbidden_app(id int). Ожидается отказ в соответствующем праве. Если действие неожиданно выполнено, зафиксируйте результат и исправьте выдачу прав из начального административного подключения; не объявляйте такую проверку успешной.

Под app_user проверьте создание серверной учётной записи командой именно этой СУБД:

```
CREATE USER 'forbidden_probe'@'%' IDENTIFIED BY 'Probe-Only-7Q9!23';
```

Нужен отказ в полномочиях, а не ошибка пароля, синтаксиса или уже существующего имени. Не проверяйте запрет командой другого движка. Списки системных схем не обязаны выглядеть одинаково; вывод делается по нужной операции.

Маршрут / MySQL 8 · Linux

# 3. Ручная копия и отдельное восстановление

11

### Сохраните исходные значения

После проверки прав повторите COUNT(*), SUM(id), SUM(amount) по трём таблицам и сохраните вывод как rows-before.txt. Зафиксируйте точное имя исходной базы. Первый опыт не требует повреждения этой базы.

12

## Сохраните дамп в точный файл

В терминале ОС вызовите mysqldump, подставив исходную базу. --result-file сохраняет SQL напрямую; в PowerShell не используется перенаправление >, которое в старых версиях меняет кодировку текста. Не добавляйте --databases: в учебном дампе не должно быть переключения обратно на исходное имя базы.

```
mysqldump -h 127.0.0.1 -P 3306 -u admin -p --single-transaction --no-tablespaces
--set-gtid-purged=OFF --default-character-set=utf8mb4 --result-file=manual.sql
pm07_demo
```

13

## Загрузите дамп только в _restore

Войдите клиентом mysql под admin. В приглашении SQL выберите USE pm07_demo_restore; проверьте SELECT DATABASE(); и выполните SOURCE с полным точным путём к manual.sql. Звёздочки и имя файла с примерной датой здесь не используются. Исходная база при этом не очищается.

Посчитайте SHA-256: sha256sum manual.sql на Linux или Get-FileHash -LiteralPath .\manual.sql -Algorithm SHA256 на Windows. Затем выполните три контрольных запроса в _restore. Если восстанавливается не текст SQL, а архив PostgreSQL, вы выбрали другой маршрут.

14

## Сверьте три предметные таблицы

Запишите rows-after.txt рядом с rows-before.txt. Сопоставьте имя таблицы, количество строк, сумму id и сумму amount; проверьте также одну конкретную запись по id=1. При различии проверьте выбранный файл и его время, список таблиц, базу назначения и первое сообщение восстановления. Только после совпадения переходите к расписанию.

Маршрут / MySQL 8 · Linux

# 4. Операция своего варианта

15

## Как сохранить файлы в терминале

Из рабочей папки выполните nano config.env, вставьте блок ниже и замените NN и параметры. В nano сохраните Ctrl+O, Enter, затем Ctrl+X. Аналогично nano run.sh открывает файл каркаса. Тройные обратные кавычки оформления внутрь файлов не копируются. При переносе из Windows сохраняйте run.sh с переводами строк LF.

Создайте папку командой mkdir -p auth. Команда nano auth/client.cnf открывает нужный парольный профиль именно этого маршрута.

16

## Подготовьте параметры и профиль клиента

Сохраните config.env. Имена базы и пути к утилитам возьмите из своего паспорта. WORK задаёт каркас до чтения конфига; вручную запускать config.env как отдельный скрипт не требуется.

```
DB="pm07_vNN"
CLIENT="mysql"
DUMP="mysqldump"
AUTH="$WORK/auth/client.cnf"
AS_OF="2026-11-04"
```

В auth/client.cnf сохраните текст ниже. OWN_PASSWORD заменяется своим учебным паролем, адрес и порт — значениями паспорта:

```
[client]
user=admin
password="OWN_PASSWORD"
host=127.0.0.1
port=3306
```

В значении password внутренняя двойная кавычка записывается как "; обратный слеш — как \. Кавычки вокруг всего значения сохраняются.

17

## Подготовьте параметры и профиль клиента · продолжение

Выполните chmod 600 auth/client.cnf. Профиль не включается в репозиторий и снимки отчёта. После подстановки своей функции выполните ручной запуск каркаса из следующего шага: он использует этот профиль и должен завершиться без запроса пароля. Только после этого настраивается cron.

18

## Сохраните каркас run.sh

Скопируйте блок целиком. Служебные функции подключения уже соответствуют выбранной СУБД. В следующем действии заменяется только run_task.

```
#!/usr/bin/env bash
set -Eeuo pipefail
umask 077
WORK="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)"
source "$WORK/config.env"
mkdir -p "$WORK/out" "$WORK/backups" "$WORK/logs"
LOG="$WORK/logs/run.log"
STAMP="$(date +%Y%m%d_%H%M%S)"
log() { printf '%s [%s] %s\n' "$(date -Is)" "$1" "$2" >> "$LOG"; }
trap 'rc=$?; log ERROR "exit=$rc"; exit "$rc"' ERR

db_query() {
    "$CLIENT" --defaults-file="$AUTH" --batch --raw --skip-column-names --
database="$DB" --execute="$1"
}
dump_db() {
    "$DUMP" --defaults-file="$AUTH" --single-transaction --no-tablespaces --set-
gtid-purged=OFF --default-character-set=utf8mb4 --result-file="$1" "$DB"
}
```

19

## Сохраните каркас run.sh · продолжение

```
run_task() {
    log INFO "Замените эту функцию целиком функцией своей карточки"
    return 2
}

log INFO "START db=$DB"
run_task
log INFO "END result=OK"
exit 0
```

20

## Ручной запуск и проверка отказа

Сохраните run.sh в рабочей папке как UTF-8 с переводами строк LF. Сначала замените функцию run_task целиком функцией своей карточки. Незаменённый каркас специально завершается ошибкой: это не выполненный вариант. Параметры находятся в config.env; пароль — в отдельном файле auth, который не включается в репозиторий.

```
cd "$HOME/pm07"
bash ./run.sh
printf 'exit=%s\n' "$?"
tail -n 8 logs/run.log
ls -l out backups
```

Ожидаются код 0, новая пара START/END и результат из карточки. Подготовьте проверку отказа в отдельной копии. Варианты 02, 05, 06 и 17 используют условие отказа из своей карточки. Для операций с БД используйте несуществующую базу в тестовой конфигурации. Сверьте ненулевой код и свежую строку ERROR.

21

## Ближайший запуск cron

На учебной Linux-машине под тем же пользователем выполните crontab -e. Если редактор выбирается впервые, выберите nano. В конце добавьте две строки; в имени рабочей папки не должно быть пробелов. $HOME задаётся cron для владельца этой записи, поэтому профиль другого пользователя здесь не используется.

```
SHELL=/bin/bash
*/2 * * * * /bin/bash "$HOME/pm07/run.sh" >> "$HOME/pm07/logs/cron.log" 2>&1
```

В nano сохраните Ctrl+O, Enter, затем Ctrl+X. Это запуск в каждую чётную минуту, а не «ровно через две минуты». Сохраните crontab -l, запишите ближайшее подходящее время, дождитесь его и сравните новую запись в logs/run.log с результатом операции. logs/cron.log принимает только обычный вывод и ошибки; если скрипт пишет всё в run.log, cron.log может остаться пустым. Тип журнала указан в паспорте. Для systemd выполните journalctl -u cron --since "10 minutes ago" (либо выданное имя crond). Для выданного текстового /var/log/syslog выполните grep CRON /var/log/syslog | tail -n 10. Найдите CMD со своим run.sh и назначенным временем. Доступ к этому журналу должен быть выдан заранее; отсутствие файла syslog не доказывает отсутствие запуска.

22

## Ближайший запуск cron · продолжение

Для варианта 06 требуется учётная запись ОС, которой на выделенной VM разрешён запуск нужной службы. Её выдаёт преподаватель до проверки; остальные варианты используют выданную обычную учётную запись. Рабочая папка и профиль клиента принадлежат фактическому владельцу задания.

23

### Постоянная периодичность

После проверочного запуска замените тестовую строку единственной постоянной. Ежедневный запуск: 0 3 * * *; еженедельный: 0 3 * * 1; ежечасный: 0 * * * *. Остальная часть команды остаётся прежней. При отсутствии частоты в карточке выберите её и объясните. Сохраните окончательный crontab -l; две параллельные записи для одного скрипта не нужны.

Маршрут / MySQL 8 · Linux

# Итог выбранного маршрута

24

В своей карточке найдите функцию операции и её обязательный опыт. Выполните только её, добавьте выбранное индивидуальное дополнение, затем заполните журнал: подключение, права, ручная копия, отдельное восстановление, ручной запуск, контролируемый отказ и действительный запуск планировщика.

Сдаваемая инструкция своего репозитория позволяет повторить запуск с выданными параметрами без пересоздания производственной модели и без чтения другого маршрута. Снимки и результаты отчёта принадлежат своей работе; приведённые ожидаемые значения не заменяют наблюдения.

Маршрут / MariaDB 10 · Linux

# Прежде чем копировать команды

1

## Работайте только в этом маршруте

ОС и СУБД должны совпадать с карточкой варианта. Здесь все команды ОС выполняются в: **терминал учебной Linux-машины**. SQL выполняется отдельно в клиенте **mariadb**. Адрес клиента на хосте и адрес внутри VM могут различаться: перепишите оба из паспорта учебной среды. Настройка сервера и канал доступа уже подготовлены; установка из другого маршрута не выполняется.

2

## Заполните подстановки

| В образце | В своём варианте |
|---|---|
| pm07_demo | pm07_vNN — имя исходной базы |
| pm07_demo_restore | pm07_vNN_restore — отдельная проверочная база |
| devices, locations, checks | Три имени таблиц из своей карточки |
| input.sql | Скачанный SQL своего варианта, сохранённый под этим рабочим именем |
| ADMIN_PASSWORD, APP_PASSWORD, READ_PASSWORD | Собственные учебные пароли, подставляемые до выполнения SQL |

В SQL сохраняйте обозначение имени таблицы: обратные кавычки вокруг имени для MySQL/MariaDB. Заменяется имя внутри них. Это важно, например, для исходной таблицы groups.

3

### Заполните подстановки · продолжение

Во всех блоках SQL сначала замените длинное имя pm07_demo_restore, затем pm07_demo. Сохраняйте каждый изменённый блок в текстовом редакторе до запуска. Пароль в SQL-строке заключён в одинарные кавычки; внутреннюю одинарную кавычку удваивают. При подготовке профиля клиента учитывайте его собственное экранирование. Пароли не помещаются в снимки отчёта.

Маршрут / MariaDB 10 · Linux

# 1. Подключение и исходные данные

4

### Подготовьте рабочую папку

```
mkdir -p "$HOME/pm07"
cd "$HOME/pm07"
pwd
```

Сохраните в этой папке SQL своей карточки под именем input.sql. Если файл скачан на другой компьютер, выполните перенос и проверку файла. Разбор использует pm07_demo; в своих командах укажите имя базы по номеру варианта.

5

## Войдите начальным администратором

Используйте выданную команду входа в mariadb. При парольной TCP-аутентификации пример в терминале: `mariadb -h 127.0.0.1 -P 3306 -u root -p`. Если в паспорте указан локальный socket-вход, используйте именно его. Создание базы и всех учётных записей ниже выполняется начальным администратором сервера, а не пользователем admin одной базы.

В отдельной учебной VM вход для выданных клиентов заранее ограничен подготовленным каналом. В примере имена MySQL-учётных записей имеют host-часть %, чтобы одна учебная роль могла пройти локальную и выданную клиентскую проверку. Эта часть — условие сопоставления клиента, а не команда настройки сетевого доступа.

```
SELECT VERSION(), CURRENT_USER(), DATABASE();
CREATE DATABASE pm07_demo CHARACTER SET utf8mb4;
CREATE DATABASE pm07_demo_restore CHARACTER SET utf8mb4;
CREATE USER 'admin'@'%' IDENTIFIED BY 'ADMIN_PASSWORD';
CREATE USER 'app_user'@'%' IDENTIFIED BY 'APP_PASSWORD';
CREATE USER 'readonly'@'%' IDENTIFIED BY 'READ_PASSWORD';
GRANT ALL PRIVILEGES ON pm07_demo.* TO 'admin'@'%';
GRANT ALL PRIVILEGES ON pm07_demo_restore.* TO 'admin'@'%';
GRANT SELECT,INSERT,UPDATE,DELETE ON pm07_demo.* TO 'app_user'@'%';
GRANT SELECT ON pm07_demo.* TO 'readonly'@'%';
SHOW GRANTS FOR 'admin'@'%';
SHOW GRANTS FOR 'app_user'@'%';
SHOW GRANTS FOR 'readonly'@'%';
```

6

## Загрузите выданный SQL от admin

Завершите соединение командой quit и войдите заново как admin в исходную базу. Полный путь к клиенту Windows берётся из паспорта. В консоли SQL выберите свою базу через USE, затем выполните SOURCE с полным путём к input.sql. На Windows путь записывается C:/Practice07/input.sql; на Linux — значение pwd и /input.sql. Это команда клиента SQL, а не строка PowerShell.

GRANT на pm07_demo.* не выдаёт глобальную привилегию CREATE USER. В этом маршруте серверные учётные записи уже созданы начальным администратором; проверка admin относится к своей базе. Дополнительный FLUSH PRIVILEGES после CREATE USER и GRANT не требуется.

7

## Проверьте исходный набор и графический клиент

Подключитесь к исходной базе как admin и выполните для каждой своей предметной таблицы (в примере devices):

```
SELECT COUNT(*) AS row_count, SUM(id) AS id_sum, SUM(amount) AS amount_sum FROM `devices`;
```

Ожидаются 4, 10, 100. Сохраните три результата с именами таблиц. access_probe содержит id=1 и value=10 и в эти три предметных результата не входит.

8

## Проверьте исходный набор и графический клиент · продолжение

На хосте откройте выданный графический клиент. В DBeaver: «База данных» (Database) → «Новое соединение» (New Database Connection) → драйвер своей СУБД → «Далее» (Next); введите адрес и порт проброса **из паспорта хоста**, имя исходной базы, readonly и его пароль → «Тест соединения» (Test Connection) → «Готово» (Finish). Если отключено «Сохранять пароль», введите пароль в появившемся отдельном окне авторизации; поле пароля в основной форме при этом недоступно. Если клиент просит драйвер, используйте выданный заранее настроенный клиент; загрузка неизвестного драйвера не заменяет подготовку среды. Откройте «Редактор SQL» (SQL Editor) → «Open SQL console» (Ctrl+Alt+Enter) и выполните SELECT COUNT(*) для одной своей таблицы. Пошаговый разбор со снимками находится в общем шаге подключения. В SSMS аналогично выбираются выданный сервер и SQL Server Authentication с readonly, затем New Query и своя база. Результат должен совпасть с CLI. 127.0.0.1 внутри VM и на хосте означает разные машины; порт из гостевого примера не переносится в хост автоматически.

# 2. Три отдельных подключения и права

9

## Позитивные опыты

Для каждого опыта создайте новое соединение и проверьте имя текущей роли. Под admin выполните создание и удаление только учебной check_admin. Под app_user измените отдельную строку access_probe и затем верните её в исходное состояние:

```
CREATE TABLE check_admin (id int);
DROP TABLE check_admin;
```

```
INSERT INTO access_probe(id,value) VALUES(9001,20);
UPDATE access_probe SET value=30 WHERE id=9001;
SELECT * FROM access_probe WHERE id=9001;
DELETE FROM access_probe WHERE id=9001;
```

Первый блок — только admin; второй — только app_user. Затем readonly читает SELECT * FROM access_probe. После опыта остаётся ровно исходная строка id=1, value=10.

10

### Негативные опыты

Под readonly отдельно проверьте INSERT в access_probe и DELETE строки id=1. Под app_user отдельно проверьте CREATE TABLE forbidden_app(id int). Ожидается отказ в соответствующем праве. Если действие неожиданно выполнено, зафиксируйте результат и исправьте выдачу прав из начального административного подключения; не объявляйте такую проверку успешной.

Под app_user проверьте создание серверной учётной записи командой именно этой СУБД:

```
CREATE USER 'forbidden_probe'@'%' IDENTIFIED BY 'Probe-Only-7Q9!23';
```

Нужен отказ в полномочиях, а не ошибка пароля, синтаксиса или уже существующего имени. Не проверяйте запрет командой другого движка. Списки системных схем не обязаны выглядеть одинаково; вывод делается по нужной операции.

Маршрут / MariaDB 10 · Linux

# 3. Ручная копия и отдельное восстановление

11

### Сохраните исходные значения

После проверки прав повторите COUNT(*), SUM(id), SUM(amount) по трём таблицам и сохраните вывод как rows-before.txt. Зафиксируйте точное имя исходной базы. Первый опыт не требует повреждения этой базы.

12

### Сохраните дамп в точный файл

В терминале ОС вызовите mariadb-dump, подставив исходную базу. --result-file сохраняет SQL напрямую; в PowerShell не используется перенаправление >, которое в старых версиях меняет кодировку текста. Не добавляйте --databases: в учебном дампе не должно быть переключения обратно на исходное имя базы.

```
mariadb-dump -h 127.0.0.1 -P 3306 -u admin -p --single-transaction --default-character-set=utf8mb4 --result-file=manual.sql pm07_demo
```

13

### Загрузите дамп только в _restore

Войдите клиентом mariadb под admin. В приглашении SQL выберите USE pm07_demo_restore; проверьте SELECT DATABASE(); и выполните SOURCE с полным точным путём к manual.sql. Звёздочки и имя файла с примерной датой здесь не используются. Исходная база при этом не очищается.

Посчитайте SHA-256: sha256sum manual.sql на Linux или Get-FileHash -LiteralPath .\manual.sql -Algorithm SHA256 на Windows. Затем выполните три контрольных запроса в _restore. Если восстанавливается не текст SQL, а архив PostgreSQL, вы выбрали другой маршрут.

14

### Сверьте три предметные таблицы

Запишите rows-after.txt рядом с rows-before.txt. Сопоставьте имя таблицы, количество строк, сумму id и сумму amount; проверьте также одну конкретную запись по id=1. При различии проверьте выбранный файл и его время, список таблиц, базу назначения и первое сообщение восстановления. Только после совпадения переходите к расписанию.

Маршрут / MariaDB 10 · Linux

# 4. Операция своего варианта

15

## Как сохранить файлы в терминале

Из рабочей папки выполните nano config.env, вставьте блок ниже и замените NN и параметры. В nano сохраните Ctrl+O, Enter, затем Ctrl+X. Аналогично nano run.sh открывает файл каркаса. Тройные обратные кавычки оформления внутрь файлов не копируются. При переносе из Windows сохраняйте run.sh с переводами строк LF.

Создайте папку командой mkdir -p auth. Команда nano auth/client.cnf открывает нужный парольный профиль именно этого маршрута.

16

## Подготовьте параметры и профиль клиента

Сохраните config.env. Имена базы и пути к утилитам возьмите из своего паспорта. WORK задаёт каркас до чтения конфига; вручную запускать config.env как отдельный скрипт не требуется.

```
DB="pm07_vNN"
CLIENT="mariadb"
DUMP="mariadb-dump"
AUTH="$WORK/auth/client.cnf"
AS_OF="2026-11-04"
ERROR_LOG="$WORK/input-error.log"
DB_SERVICE="mariadb"
```

В auth/client.cnf сохраните текст ниже. OWN_PASSWORD заменяется своим учебным паролем, адрес и порт — значениями паспорта:

```
[client]
user=admin
password="OWN_PASSWORD"
host=127.0.0.1
port=3306
```

В значении password внутренняя двойная кавычка записывается как "; обратный слеш — как \. Кавычки вокруг всего значения сохраняются.

17

## Подготовьте параметры и профиль клиента · продолжение

ERROR_LOG нужен только варианту 17: сохраните выданный в его карточке учебный журнал как input-error.log в рабочую папку. DB_SERVICE нужен только варианту 06; имя службы и право её запуска выдаются вместе со средой.

Выполните chmod 600 auth/client.cnf. Профиль не включается в репозиторий и снимки отчёта. После подстановки своей функции выполните ручной запуск каркаса из следующего шага: он использует этот профиль и должен завершиться без запроса пароля. Только после этого настраивается cron.

18

## Сохраните каркас run.sh

Скопируйте блок целиком. Служебные функции подключения уже соответствуют выбранной СУБД. В следующем действии заменяется только run_task.

```
#!/usr/bin/env bash
set -Eeuo pipefail
umask 077
WORK="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)"
source "$WORK/config.env"
mkdir -p "$WORK/out" "$WORK/backups" "$WORK/logs"
LOG="$WORK/logs/run.log"
STAMP="$(date +%Y%m%d_%H%M%S)"
log() { printf '%s [%s] %s\n' "$(date -Is)" "$1" "$2" >> "$LOG"; }
trap 'rc=$?; log ERROR "exit=$rc"; exit "$rc"' ERR

db_query() {
    "$CLIENT" --defaults-file="$AUTH" --batch --raw --skip-column-names --
database="$DB" --execute="$1"
}
dump_db() {
    "$DUMP" --defaults-file="$AUTH" --single-transaction --default-character-
set=utf8mb4 --result-file="$1" "$DB"
}
```

19

## Сохраните каркас run.sh · продолжение

```
run_task() {
    log INFO "Замените эту функцию целиком функцией своей карточки"
    return 2
}

log INFO "START db=$DB"
run_task
log INFO "END result=OK"
exit 0
```

20

## Ручной запуск и проверка отказа

Сохраните run.sh в рабочей папке как UTF-8 с переводами строк LF. Сначала замените функцию run_task целиком функцией своей карточки. Незаменённый каркас специально завершается ошибкой: это не выполненный вариант. Параметры находятся в config.env; пароль — в отдельном файле auth, который не включается в репозиторий.

```
cd "$HOME/pm07"
bash ./run.sh
printf 'exit=%s\n' "$?"
tail -n 8 logs/run.log
ls -l out backups
```

Ожидаются код 0, новая пара START/END и результат из карточки. Подготовьте проверку отказа в отдельной копии. Варианты 02, 05, 06 и 17 используют условие отказа из своей карточки. Для операций с БД используйте несуществующую базу в тестовой конфигурации. Сверьте ненулевой код и свежую строку ERROR.

21

## Ближайший запуск cron

На учебной Linux-машине под тем же пользователем выполните crontab -e. Если редактор выбирается впервые, выберите nano. В конце добавьте две строки; в имени рабочей папки не должно быть пробелов. $HOME задаётся cron для владельца этой записи, поэтому профиль другого пользователя здесь не используется.

```
SHELL=/bin/bash
*/2 * * * * /bin/bash "$HOME/pm07/run.sh" >> "$HOME/pm07/logs/cron.log" 2>&1
```

В nano сохраните Ctrl+O, Enter, затем Ctrl+X. Это запуск в каждую чётную минуту, а не «ровно через две минуты». Сохраните crontab -l, запишите ближайшее подходящее время, дождитесь его и сравните новую запись в logs/run.log с результатом операции. logs/cron.log принимает только обычный вывод и ошибки; если скрипт пишет всё в run.log, cron.log может остаться пустым. Тип журнала указан в паспорте. Для systemd выполните journalctl -u cron --since "10 minutes ago" (либо выданное имя crond). Для выданного текстового /var/log/syslog выполните grep CRON /var/log/syslog | tail -n 10. Найдите CMD со своим run.sh и назначенным временем. Доступ к этому журналу должен быть выдан заранее; отсутствие файла syslog не доказывает отсутствие запуска.

22

## Ближайший запуск cron · продолжение

Для варианта 06 требуется учётная запись ОС, которой на выделенной VM разрешён запуск нужной службы. Её выдаёт преподаватель до проверки; остальные варианты используют выданную обычную учётную запись. Рабочая папка и профиль клиента принадлежат фактическому владельцу задания.

23

### Постоянная периодичность

После проверочного запуска замените тестовую строку единственной постоянной. Ежедневный запуск: 0 3 * * *; еженедельный: 0 3 * * 1; ежечасный: 0 * * * *. Остальная часть команды остаётся прежней. При отсутствии частоты в карточке выберите её и объясните. Сохраните окончательный crontab -l; две параллельные записи для одного скрипта не нужны.

Маршрут / MariaDB 10 · Linux

# Итог выбранного маршрута

24

В своей карточке найдите функцию операции и её обязательный опыт. Выполните только её, добавьте выбранное индивидуальное дополнение, затем заполните журнал: подключение, права, ручная копия, отдельное восстановление, ручной запуск, контролируемый отказ и действительный запуск планировщика.

Сдаваемая инструкция своего репозитория позволяет повторить запуск с выданными параметрами без пересоздания производственной модели и без чтения другого маршрута. Снимки и результаты отчёта принадлежат своей работе; приведённые ожидаемые значения не заменяют наблюдения.

Маршрут / PostgreSQL 15/16 · Linux

# Прежде чем копировать команды

1

## Работайте только в этом маршруте

ОС и СУБД должны совпадать с карточкой варианта. Здесь все команды ОС выполняются в: **терминал учебной Linux-машины**. SQL выполняется отдельно в клиенте **psql**. Адрес клиента на хосте и адрес внутри VM могут различаться: перепишите оба из паспорта учебной среды. Настройка сервера и канал доступа уже подготовлены; установка из другого маршрута не выполняется.

2

## Заполните подстановки

| В образце | В своём варианте |
|---|---|
| pm07_demo | pm07_vNN — имя исходной базы |
| pm07_demo_restore | pm07_vNN_restore — отдельная проверочная база |
| devices, locations, checks | Три имени таблиц из своей карточки |
| input.sql | Скачанный SQL своего варианта, сохранённый под этим рабочим именем |
| ADMIN_PASSWORD, APP_PASSWORD, READ_PASSWORD | Собственные учебные пароли, подставляемые до выполнения SQL |

В SQL сохраняйте обозначение имени таблицы: двойные кавычки вокруг имени для PostgreSQL. Заменяется имя внутри них. Это важно, например, для исходной таблицы groups.

3

### Заполните подстановки · продолжение

Во всех блоках SQL сначала замените длинное имя pm07_demo_restore, затем pm07_demo. Сохраняйте каждый изменённый блок в текстовом редакторе до запуска. Пароль в SQL-строке заключён в одинарные кавычки; внутреннюю одинарную кавычку удваивают. При подготовке профиля клиента учитывайте его собственное экранирование. Пароли не помещаются в снимки отчёта.

Маршрут / PostgreSQL 15/16 · Linux

# 1. Подключение и исходные данные

4

### Подготовьте рабочую папку

```
mkdir -p "$HOME/pm07"
cd "$HOME/pm07"
pwd
```

Сохраните в этой папке SQL своей карточки под именем input.sql. Если файл скачан на другой компьютер, выполните перенос и проверку файла. Разбор использует pm07_demo; в своих командах укажите имя базы по номеру варианта.

5

## Войдите начальным администратором

Откройте psql из выданного каталога клиента. Подключитесь к базе postgres от начальной роли postgres; если роль называется иначе, используйте паспорт. Пример параметров: host 127.0.0.1, port 5432. После запроса введите пароль интерактивно. В клиенте выполните следующий SQL после подстановок:

```
SELECT version(), current_user, current_database();
CREATE ROLE admin LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE PASSWORD
'ADMIN_PASSWORD';
CREATE ROLE app_user LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE PASSWORD
'APP_PASSWORD';
CREATE ROLE readonly LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE PASSWORD
'READ_PASSWORD';
CREATE DATABASE pm07_demo OWNER admin TEMPLATE template0 ENCODING 'UTF8';
CREATE DATABASE pm07_demo_restore OWNER admin TEMPLATE template0 ENCODING
'UTF8';
```

6

## Загрузите выданный SQL от admin

Завершите текущее соединение командой \q. Подключитесь admin к своей исходной базе. Пример команды в терминале ОС:

```
psql -h 127.0.0.1 -p 5432 -U admin -d pm07_demo -v ON_ERROR_STOP=1
```

В приглашении psql выполните `\i` и полный путь к input.sql. На Windows используйте путь C:/Practice07/input.sql; на Linux — путь, показанный pwd, с /input.sql в конце. Затем выполните:

```
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO app_user, readonly;
GRANT SELECT,INSERT,UPDATE,DELETE ON ALL TABLES IN SCHEMA public TO app_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;
GRANT USAGE,SELECT ON ALL SEQUENCES IN SCHEMA public TO app_user;
ALTER DEFAULT PRIVILEGES FOR ROLE admin IN SCHEMA public
  GRANT SELECT,INSERT,UPDATE,DELETE ON TABLES TO app_user;
ALTER DEFAULT PRIVILEGES FOR ROLE admin IN SCHEMA public
  GRANT SELECT ON TABLES TO readonly;
```

7

## Загрузите выданный SQL от admin · продолжение

Таблицы создаёт admin, поэтому и правила для будущих таблиц заданы для этой роли. CREATEROLE и SUPERUSER ему для операций своей базы не выдаются. Служебный postgresql.service иногда показывает active (exited); работоспособность конкретного кластера проверяется реальным подключением и SELECT version(), а не одной строкой агрегирующей службы.

8

## Проверьте исходный набор и графический клиент

Подключитесь к исходной базе как admin и выполните для каждой своей предметной таблицы (в примере devices):

```
SELECT COUNT(*) AS row_count, SUM(id) AS id_sum, SUM(amount) AS amount_sum FROM "devices";
```

Ожидаются 4, 10, 100. Сохраните три результата с именами таблиц. access_probe содержит id=1 и value=10 и в эти три предметных результата не входит.

9

## Проверьте исходный набор и графический клиент · продолжение

На хосте откройте выданный графический клиент. В DBeaver: «База данных» (Database) → «Новое соединение» (New Database Connection) → драйвер своей СУБД → «Далее» (Next); введите адрес и порт проброса **из паспорта хоста**, имя исходной базы, readonly и его пароль → «Тест соединения» (Test Connection) → «Готово» (Finish). Если отключено «Сохранять пароль», введите пароль в появившемся отдельном окне авторизации; поле пароля в основной форме при этом недоступно. Если клиент просит драйвер, используйте выданный заранее настроенный клиент; загрузка неизвестного драйвера не заменяет подготовку среды. Откройте «Редактор SQL» (SQL Editor) → «Open SQL console» (Ctrl+Alt+Enter) и выполните SELECT COUNT(*) для одной своей таблицы. Пошаговый разбор со снимками находится в общем шаге подключения. В SSMS аналогично выбираются выданный сервер и SQL Server Authentication с readonly, затем New Query и своя база. Результат должен совпасть с CLI. 127.0.0.1 внутри VM и на хосте означает разные машины; порт из гостевого примера не переносится в хост автоматически.

# 2. Три отдельных подключения и права

10

## Позитивные опыты

Для каждого опыта создайте новое соединение и проверьте имя текущей роли. Под admin выполните создание и удаление только учебной check_admin. Под app_user измените отдельную строку access_probe и затем верните её в исходное состояние:

```
CREATE TABLE check_admin (id int);
DROP TABLE check_admin;
```

```
INSERT INTO access_probe(id,value) VALUES(9001,20);
UPDATE access_probe SET value=30 WHERE id=9001;
SELECT * FROM access_probe WHERE id=9001;
DELETE FROM access_probe WHERE id=9001;
```

Первый блок — только admin; второй — только app_user. Затем readonly читает SELECT * FROM access_probe. После опыта остаётся ровно исходная строка id=1, value=10.

11

### Негативные опыты

Под readonly отдельно проверьте INSERT в access_probe и DELETE строки id=1. Под app_user отдельно проверьте CREATE TABLE forbidden_app(id int). Ожидается отказ в соответствующем праве. Если действие неожиданно выполнено, зафиксируйте результат и исправьте выдачу прав из начального административного подключения; не объявляйте такую проверку успешной.

Под app_user проверьте создание серверной учётной записи командой именно этой СУБД:

```
CREATE ROLE forbidden_probe LOGIN;
```

Нужен отказ в полномочиях, а не ошибка пароля, синтаксиса или уже существующего имени. Не проверяйте запрет командой другого движка. Списки системных схем не обязаны выглядеть одинаково; вывод делается по нужной операции.

Маршрут / PostgreSQL 15/16 · Linux

# 3. Ручная копия и отдельное восстановление

12

### Сохраните исходные значения

После проверки прав повторите COUNT(*), SUM(id), SUM(amount) по трём таблицам и сохраните вывод как rows-before.txt. Зафиксируйте точное имя исходной базы. Первый опыт не требует повреждения этой базы.

13

## Создайте архив и восстановите пустую копию

В терминале ОС выполните pg_dump от admin с запросом пароля. Полные пути Windows к pg_dump.exe и pg_restore.exe берутся из того же установленного комплекта, что и psql. В примере показаны аргументы утилит; на Windows перед путём в кавычках ставится оператор &.

```
pg_dump -h 127.0.0.1 -p 5432 -U admin -d pm07_demo -Fc -f manual.dump
pg_restore --list manual.dump
pg_restore -h 127.0.0.1 -p 5432 -U admin -d pm07_demo_restore --no-owner --no-
privileges --exit-on-error manual.dump
```

Восстановление направлено в уже созданную пустую _restore; --create не используется. Вход в другую базу того же сервера выполняется отдельным подключением. Запись вида INSERT INTO исходная.public.table SELECT FROM другая_база.public.table не является переносом между базами PostgreSQL.

Посчитайте SHA-256 manual.dump: sha256sum manual.dump на Linux или Get-FileHash -LiteralPath .\manual.dump -Algorithm SHA256 на Windows. Сохраните значение. Подключитесь к _restore и выполните три контрольных запроса. Список архива не заменяет это сравнение.

14

## Проверка другого архива

Для нового или частичного архива используйте ещё одну пустую проверочную базу. Войдите начальной ролью postgres к базе postgres и выполните после замены имени:

```
CREATE DATABASE pm07_demo_restore2 OWNER admin TEMPLATE template0 ENCODING
'UTF8';
```

Первую _restore и исходную базу сохраните. Завершите это соединение. В терминале восстановите новый архив от admin; его точный путь заменяет manual.dump:

```
pg_restore -h 127.0.0.1 -p 5432 -U admin -d pm07_demo_restore2 --no-owner --no-
privileges --exit-on-error manual.dump
```

При повторе начальный администратор выделяет следующее уникальное пустое имя. Имя _restore2 используется только как назначение; непустая база без объяснения не очищается. Для частичной копии сравните только включённые таблицы и подтвердите отсутствие остальных.

15

### Сверьте три предметные таблицы

Запишите rows-after.txt рядом с rows-before.txt. Сопоставьте имя таблицы, количество строк, сумму id и сумму amount; проверьте также одну конкретную запись по id=1. При различии проверьте выбранный файл и его время, список таблиц, базу назначения и первое сообщение восстановления. Только после совпадения переходите к расписанию.

Маршрут / PostgreSQL 15/16 · Linux

# 4. Операция своего варианта

16

### Как сохранить файлы в терминале

Из рабочей папки выполните nano config.env, вставьте блок ниже и замените NN и параметры. В nano сохраните Ctrl+O, Enter, затем Ctrl+X. Аналогично nano run.sh открывает файл каркаса. Тройные обратные кавычки оформления внутрь файлов не копируются. При переносе из Windows сохраняйте run.sh с переводами строк LF.

Создайте папку командой mkdir -p auth. Команда nano auth/pgpass открывает нужный парольный профиль именно этого маршрута.

17

## Подготовьте параметры и профиль клиента

Сохраните config.env. Имена базы и пути к утилитам возьмите из своего паспорта. WORK задаёт каркас до чтения конфига; вручную запускать config.env как отдельный скрипт не требуется.

```
DB="pm07_vNN"
DB_HOST="127.0.0.1"
DB_PORT="5432"
CLIENT="psql"
DUMP="pg_dump"
AUTH="$WORK/auth/pgpass"
AS_OF="2026-11-04"
CHECK_TIME="2026-11-04 09:30:00"
```

AS_OF используется в варианте 10, CHECK_TIME — в варианте 09. В auth/pgpass сохраните одну строку, заменив NN, адрес, порт и OWN_PASSWORD:

```
127.0.0.1:5432:pm07_vNN:admin:OWN_PASSWORD
```

Двоеточие внутри значения записывается как :; обратный слеш — как \. Разделительные двоеточия между пятью полями не экранируются.

18

## Подготовьте параметры и профиль клиента · продолжение

Выполните chmod 600 auth/pgpass. Профиль не включается в репозиторий и снимки отчёта. После подстановки своей функции выполните ручной запуск каркаса из следующего шага: он использует этот профиль и должен завершиться без запроса пароля. Только после этого настраивается cron.

19

## Сохраните каркас run.sh

Скопируйте блок целиком. Служебные функции подключения уже соответствуют выбранной СУБД. В следующем действии заменяется только run_task.

```
#!/usr/bin/env bash
set -Eeuo pipefail
umask 077
WORK="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)"
source "$WORK/config.env"
mkdir -p "$WORK/out" "$WORK/backups" "$WORK/logs"
LOG="$WORK/logs/run.log"
STAMP="$(date +%Y%m%d_%H%M%S)"
log() { printf '%s [%s] %s\n' "$(date -Is)" "$1" "$2" >> "$LOG"; }
trap 'rc=$?; log ERROR "exit=$rc"; exit "$rc"' ERR

export PGPASSFILE="$AUTH"
db_query() {
    "$CLIENT" -X -w -h "$DB_HOST" -p "$DB_PORT" -U admin -d "$DB" -v
ON_ERROR_STOP=1 -A -t -c "$1"
}
dump_db() {
    "$DUMP" -w -h "$DB_HOST" -p "$DB_PORT" -U admin -d "$DB" -Fc -f "$1"
}
```

20

## Сохраните каркас run.sh · продолжение

```
run_task() {
    log INFO "Замените эту функцию целиком функцией своей карточки"
    return 2
}

log INFO "START db=$DB"
run_task
log INFO "END result=OK"
exit 0
```

21

## Ручной запуск и проверка отказа

Сохраните run.sh в рабочей папке как UTF-8 с переводами строк LF. Сначала замените функцию run_task целиком функцией своей карточки. Незаменённый каркас специально завершается ошибкой: это не выполненный вариант. Параметры находятся в config.env; пароль — в отдельном файле auth, который не включается в репозиторий.

```
cd "$HOME/pm07"
bash ./run.sh
printf 'exit=%s\n' "$?"
tail -n 8 logs/run.log
ls -l out backups
```

Ожидаются код 0, новая пара START/END и результат из карточки. Подготовьте проверку отказа в отдельной копии. Варианты 02, 05, 06 и 17 используют условие отказа из своей карточки. Для операций с БД используйте несуществующую базу в тестовой конфигурации. Сверьте ненулевой код и свежую строку ERROR.

22

## Ближайший запуск cron

На учебной Linux-машине под тем же пользователем выполните crontab -e. Если редактор выбирается впервые, выберите nano. В конце добавьте две строки; в имени рабочей папки не должно быть пробелов. $HOME задаётся cron для владельца этой записи, поэтому профиль другого пользователя здесь не используется.

```
SHELL=/bin/bash
*/2 * * * * /bin/bash "$HOME/pm07/run.sh" >> "$HOME/pm07/logs/cron.log" 2>&1
```

В nano сохраните Ctrl+O, Enter, затем Ctrl+X. Это запуск в каждую чётную минуту, а не «ровно через две минуты». Сохраните crontab -l, запишите ближайшее подходящее время, дождитесь его и сравните новую запись в logs/run.log с результатом операции. logs/cron.log принимает только обычный вывод и ошибки; если скрипт пишет всё в run.log, cron.log может остаться пустым. Тип журнала указан в паспорте. Для systemd выполните journalctl -u cron --since "10 minutes ago" (либо выданное имя crond). Для выданного текстового /var/log/syslog выполните grep CRON /var/log/syslog | tail -n 10. Найдите CMD со своим run.sh и назначенным временем. Доступ к этому журналу должен быть выдан заранее; отсутствие файла syslog не доказывает отсутствие запуска.

23

### Ближайший запуск cron · продолжение

Для варианта 06 требуется учётная запись ОС, которой на выделенной VM разрешён запуск нужной службы. Её выдаёт преподаватель до проверки; остальные варианты используют выданную обычную учётную запись. Рабочая папка и профиль клиента принадлежат фактическому владельцу задания.

24

### Постоянная периодичность

После проверочного запуска замените тестовую строку единственной постоянной. Ежедневный запуск: 0 3 * * *; еженедельный: 0 3 * * 1; ежечасный: 0 * * * *. Остальная часть команды остаётся прежней. При отсутствии частоты в карточке выберите её и объясните. Сохраните окончательный crontab -l; две параллельные записи для одного скрипта не нужны.

Маршрут / PostgreSQL 15/16 · Linux

# Итог выбранного маршрута

25

В своей карточке найдите функцию операции и её обязательный опыт. Выполните только её, добавьте выбранное индивидуальное дополнение, затем заполните журнал: подключение, права, ручная копия, отдельное восстановление, ручной запуск, контролируемый отказ и действительный запуск планировщика.

Сдаваемая инструкция своего репозитория позволяет повторить запуск с выданными параметрами без пересоздания производственной модели и без чтения другого маршрута. Снимки и результаты отчёта принадлежат своей работе; приведённые ожидаемые значения не заменяют наблюдения.

Маршрут / SQL Server Express · Windows

# Прежде чем копировать команды

1

## Работайте только в этом маршруте

ОС и СУБД должны совпадать с карточкой варианта. Здесь все команды ОС выполняются в: **Windows PowerShell на учебной Windows-машине**. SQL выполняется отдельно в клиенте **sqlcmd**. Адрес клиента на хосте и адрес внутри VM могут различаться: перепишите оба из паспорта учебной среды. Настройка сервера и канал доступа уже подготовлены; установка из другого маршрута не выполняется.

2

## Заполните подстановки

| В образце | В своём варианте |
|---|---|
| pm07_demo | pm07_vNN — имя исходной базы |
| pm07_demo_restore | pm07_vNN_restore — отдельная проверочная база |
| devices, locations, checks | Три имени таблиц из своей карточки |
| input.sql | Скачанный SQL своего варианта, сохранённый под этим рабочим именем |
| ADMIN_PASSWORD, APP_PASSWORD, READ_PASSWORD | Собственные учебные пароли, подставляемые до выполнения SQL |

В SQL сохраняйте обозначение имени таблицы: квадратные скобки вокруг имени для SQL Server. Заменяется имя внутри них. Это важно, например, для исходной таблицы groups.

3

### Заполните подстановки · продолжение

Во всех блоках SQL сначала замените длинное имя pm07_demo_restore, затем pm07_demo. Сохраняйте каждый изменённый блок в текстовом редакторе до запуска. Пароль в SQL-строке заключён в одинарные кавычки; внутреннюю одинарную кавычку удваивают. При подготовке профиля клиента учитывайте его собственное экранирование. Пароли не помещаются в снимки отчёта.

Маршрут / SQL Server Express · Windows

# 1. Подключение и исходные данные

4

### Подготовьте рабочую папку

```
New-Item -ItemType Directory -Force C:\Practice07 | Out-Null
Set-Location C:\Practice07
```

Сохраните в этой папке SQL своей карточки под именем input.sql. Если файл скачан на другой компьютер, выполните перенос и проверку файла. Разбор использует pm07_demo; в своих командах укажите имя базы по номеру варианта.

5

## Войдите начальным администратором

Откройте SSMS → Connect → Database Engine. Укажите выданное имя экземпляра (в образце .\SQLEXPRESS), Authentication = Windows Authentication, затем Connect. Нажмите New Query. Следующий блок выполняется только от выданной начальной серверной учётной записи. Перед выполнением замените имена баз и три пароля.

```
SELECT @@VERSION AS server_version, ORIGINAL_LOGIN() AS login_name;
CREATE DATABASE pm07_demo;
GO
CREATE DATABASE pm07_demo_restore;
GO
CREATE LOGIN admin WITH PASSWORD='ADMIN_PASSWORD';
CREATE LOGIN app_user WITH PASSWORD='APP_PASSWORD';
CREATE LOGIN readonly WITH PASSWORD='READ_PASSWORD';
GO
USE pm07_demo;
GO
CREATE USER admin FOR LOGIN admin;
CREATE USER app_user FOR LOGIN app_user;
CREATE USER readonly FOR LOGIN readonly;
ALTER ROLE db_owner ADD MEMBER admin;
ALTER ROLE db_datareader ADD MEMBER app_user;
ALTER ROLE db_datawriter ADD MEMBER app_user;
ALTER ROLE db_datareader ADD MEMBER readonly;
```

6

## Войдите начальным администратором · продолжение

Строки GO сохраняются: SSMS и sqlcmd отправляют отдельные пакеты, чтобы база уже существовала к моменту переключения USE. GO не является SQL-командой, поэтому этот блок выполняется именно в указанном клиенте.

admin имеет права владельца своей базы, а не право создавать серверные логины. Исходная Windows-учётная запись используется для подготовки и первого восстановления отдельной базы. Для плановой операции Windows-аккаунт и доступ к каталогу выдаются в паспорте, а не выводятся из имени SQL-логина admin.

7

## Загрузите выданный SQL

В SSMS откройте input.sql через File → Open → File. В списке базы над редактором выберите свою исходную pm07_vNN, проверьте имя и нажмите Execute. Не выбирайте master и не загружайте данные в _restore. После загрузки выполните SELECT из трёх предметных таблиц. Повторная загрузка того же CREATE TABLE в непустую базу закономерно выдаст ошибку; это не повод удалять другую базу.

8

## Проверьте исходный набор и графический клиент

Подключитесь к исходной базе как admin и выполните для каждой своей предметной таблицы (в примере devices):

```
SELECT COUNT(*) AS row_count, SUM(id) AS id_sum, SUM(amount) AS amount_sum FROM [dbo].[devices];
```

Ожидаются 4, 10, 100. Сохраните три результата с именами таблиц. access_probe содержит id=1 и value=10 и в эти три предметных результата не входит.

9

## Проверьте исходный набор и графический клиент · продолжение

На хосте откройте выданный графический клиент. В DBeaver: «База данных» (Database) → «Новое соединение» (New Database Connection) → драйвер своей СУБД → «Далее» (Next); введите адрес и порт проброса **из паспорта хоста**, имя исходной базы, readonly и его пароль → «Тест соединения» (Test Connection) → «Готово» (Finish). Если отключено «Сохранять пароль», введите пароль в появившемся отдельном окне авторизации; поле пароля в основной форме при этом недоступно. Если клиент просит драйвер, используйте выданный заранее настроенный клиент; загрузка неизвестного драйвера не заменяет подготовку среды. Откройте «Редактор SQL» (SQL Editor) → «Open SQL console» (Ctrl+Alt+Enter) и выполните SELECT COUNT(*) для одной своей таблицы. Пошаговый разбор со снимками находится в общем шаге подключения. В SSMS аналогично выбираются выданный сервер и SQL Server Authentication с readonly, затем New Query и своя база. Результат должен совпасть с CLI. 127.0.0.1 внутри VM и на хосте означает разные машины; порт из гостевого примера не переносится в хост автоматически.

# 2. Три отдельных подключения и права

10

## Позитивные опыты

Для каждого опыта создайте новое соединение и проверьте имя текущей роли. Под admin выполните создание и удаление только учебной check_admin. Под app_user измените отдельную строку access_probe и затем верните её в исходное состояние:

```
CREATE TABLE check_admin (id int);
DROP TABLE check_admin;
```

```
INSERT INTO access_probe(id,value) VALUES(9001,20);
UPDATE access_probe SET value=30 WHERE id=9001;
SELECT * FROM access_probe WHERE id=9001;
DELETE FROM access_probe WHERE id=9001;
```

Первый блок — только admin; второй — только app_user. Затем readonly читает SELECT * FROM access_probe. После опыта остаётся ровно исходная строка id=1, value=10.

11

### Негативные опыты

Под readonly отдельно проверьте INSERT в access_probe и DELETE строки id=1. Под app_user отдельно проверьте CREATE TABLE forbidden_app(id int). Ожидается отказ в соответствующем праве. Если действие неожиданно выполнено, зафиксируйте результат и исправьте выдачу прав из начального административного подключения; не объявляйте такую проверку успешной.

Под app_user проверьте создание серверной учётной записи командой именно этой СУБД:

```
CREATE LOGIN forbidden_probe WITH PASSWORD='Probe-Only-7Q9!23';
```

Нужен отказ в полномочиях, а не ошибка пароля, синтаксиса или уже существующего имени. Не проверяйте запрет командой другого движка. Списки системных схем не обязаны выглядеть одинаково; вывод делается по нужной операции.

Маршрут / SQL Server Express · Windows

# 3. Ручная копия и отдельное восстановление

12

### Сохраните исходные значения

После проверки прав повторите COUNT(*), SUM(id), SUM(amount) по трём таблицам и сохраните вывод как rows-before.txt. Зафиксируйте точное имя исходной базы. Первый опыт не требует повреждения этой базы.

13

## Создайте копию из SSMS

Каталоги C:\Practice07\backups и C:\Practice07\restore-data должны быть подготовлены на машине SQL Server с правом записи для учётной записи службы SQL Server. Папка на хосте с таким же именем не является серверным каталогом. Выполните из начального административного подключения после подстановки имени базы и уникального имени файла:

```
BACKUP DATABASE pm07_demo
TO DISK=N'C:\Practice07\backups\manual.bak'
WITH INIT,CHECKSUM;
RESTORE VERIFYONLY FROM DISK=N'C:\Practice07\backups\manual.bak';
RESTORE FILELISTONLY FROM DISK=N'C:\Practice07\backups\manual.bak';
```

В Express не используется WITH COMPRESSION. В выводе FILELISTONLY найдите LogicalName строки типа D и строки типа L. Подставьте эти два имени в MOVE; названия в примере не гарантируются для другого исходного файла. Далее восстанавливается только отдельная _restore, подготовленная для проверки:

14

## Создайте копию из SSMS · продолжение

```
RESTORE DATABASE pm07_demo_restore
FROM DISK=N'C:\Practice07\backups\manual.bak'
WITH MOVE N'pm07_demo' TO N'C:\Practice07\restore-data\demo_restore.mdf',
     MOVE N'pm07_demo_log' TO N'C:\Practice07\restore-
data\demo_restore_log.ldf',
     REPLACE,RECOVERY;
```

REPLACE здесь разрешён только для заранее выделенной пустой проверочной _restore; имя исходной базы в этой команде не используется. Если соединение удерживает проверочную базу, закройте его, а не переводите исходную базу в SINGLE_USER. Для plan-восстановления варианта 20 право восстановления и владение именно этой проверочной базой заранее выдаются учётной записи задачи. Членство в db_owner само по себе не подменяет все серверные полномочия RESTORE.

На машине с доступом к серверному файлу сохраните (Get-FileHash -LiteralPath C:\Practice07\backups\manual.bak -Algorithm SHA256).Hash в журнал. Прочитайте три таблицы проверочной базы и сравните контрольные значения.

15

### Сверьте три предметные таблицы

Запишите rows-after.txt рядом с rows-before.txt. Сопоставьте имя таблицы, количество строк, сумму id и сумму amount; проверьте также одну конкретную запись по id=1. При различии проверьте выбранный файл и его время, список таблиц, базу назначения и первое сообщение восстановления. Только после совпадения переходите к расписанию.

Маршрут / SQL Server Express · Windows

# 4. Операция своего варианта

16

### Как сохранить файлы без лишнего расширения

Откройте Блокнот. Для config.ps1 и run.ps1 выберите «Сохранить как» → «Все файлы», точное имя и кодировку UTF-8 с BOM, совместимую с Windows PowerShell 5.1. Проводник → Вид → Расширения имён файлов позволяет проверить, что не получилось run.ps1.txt. Каждый блок заменяет файл целиком; не копируйте в него тройные обратные кавычки из оформления.

Для профиля создайте папку C:\Practice07\auth командой New-Item -ItemType Directory -Force. Откройте в Блокноте точный файл, указанный ниже, сохраните как «Все файлы» и подставьте свои значения. Парольный профиль — обычный закрытый файл, а не часть отчёта.

17

## Подготовьте параметры и профиль клиента

Создайте config.ps1 в C:\Practice07. Замените NN и пути примерных утилит значениями паспорта. Выполните Test-Path -LiteralPath для каждого пути к exe: ожидается True.

```
$Database = 'pm07_vNN'
$ServerInstance = '.\SQLEXPRESS'
$Client = 'C:\Program Files\Microsoft SQL Server\Client
SDK\ODBC\170\Tools\Binn\SQLCMD.EXE'
$AsOf = '2026-11-04'
$RestoreDatabase = 'pm07_vNN_restore'
$RestoreDataDir = 'C:\Practice07\restore-data'
$LogicalData = 'pm07_vNN'
$LogicalLog = 'pm07_vNN_log'
```

Windows-подключение -E использует учётную запись владельца задачи. Сервер видит путь BACKUP на своей машине. Операция 20 требует отдельно выданного права восстановления _restore; его отсутствие не исправляется сменой учётной записи на SYSTEM. LogicalData и LogicalLog заполняются по FILELISTONLY, а не по догадке.

18

## Сохраните каркас run.ps1

Скопируйте блок целиком. В следующем действии заменяется только Invoke-Task.

```
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
$Work = $PSScriptRoot
. (Join-Path $Work 'config.ps1')
foreach ($dir in 'out','backups','logs') {
    New-Item -ItemType Directory -Force -Path (Join-Path $Work $dir) | Out-Null
}
$Log = Join-Path $Work 'logs\run.log'
$Stamp = Get-Date -Format 'yyyyMMdd_HHmmss'
function Write-Log([string]$Level,[string]$Message) {
    "$(Get-Date -Format o) [$Level] $Message" | Add-Content -LiteralPath $Log -
Encoding UTF8
}

function Invoke-Db([string]$Query) {
    & $Client -S $ServerInstance -E -d $Database -b -r 1 -Q $Query
    if ($LASTEXITCODE -ne 0) { throw "sqlcmd exit=$LASTEXITCODE" }
}
function Backup-Db([string]$File) {
    Invoke-Db "BACKUP DATABASE [$Database] TO DISK=N'$File' WITH INIT,CHECKSUM;"
| Out-Null
}
```

19

## Сохраните каркас run.ps1 · продолжение

```
function Invoke-Task {
    throw 'Замените эту функцию целиком функцией своей карточки'
}
try {
    Write-Log INFO "START db=$Database"
    Invoke-Task
    Write-Log INFO 'END result=OK'
    exit 0
} catch {
    Write-Log ERROR $_.Exception.Message
    exit 1
}
```

20

## Ручной запуск и проверка отказа

Сохраните run.ps1 и config.ps1 в C:\Practice07. Замените Invoke-Task целиком функцией своей карточки. Если каркас ещё содержит throw с указанием заменить функцию, операция не выполнена. Сначала используйте ту же учётную запись Windows, от которой затем будет запускаться задача.

```
powershell.exe -NoProfile -File C:\Practice07\run.ps1
$LASTEXITCODE
Get-Content -LiteralPath C:\Practice07\logs\run.log -Tail 8
Get-ChildItem -LiteralPath C:\Practice07\out,C:\Practice07\backups
```

Ожидаются код 0, новая START/END и результат своей операции. Сохраните отдельные run-test.ps1 и config-test.ps1; в копии скрипта измените только имя подключаемого конфига. В тестовом конфиге задайте несуществующую базу, а для файловой операции — несуществующий исходный путь. Ожидаются ERROR и ненулевой код. Рабочий конфиг с паролем не перезаписывается. Если политика учебной машины запрещает локальные скрипты, используйте выданный разрешённый учебный профиль; общий ExecutionPolicy не меняется ради примера.

21

## Создание однократной задачи

Откройте Пуск → Планировщик заданий → Создать задачу. На вкладке «Общие» оставьте ту же учётную запись Windows, которая прошла ручную проверку. Для учебного опыта выберите выполнение при входе этого пользователя в систему; SYSTEM не подставляется вместо него.

На вкладке «Триггеры» нажмите «Создать», выберите «По расписанию» → «Однократно» и время на несколько минут позже текущего. На вкладке «Действия» выберите «Запуск программы» и заполните поля раздельно:

| Поле | Значение |
| --- | --- |
| Программа | C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe |
| Аргументы | -NoProfile -File C:\Practice07\run.ps1 |
| Рабочая папка | C:\Practice07 |

22

### Создание однократной задачи · продолжение

Сохраните задачу с именем PM07_VNN, заменив NN номером. Запишите назначенное время и дождитесь его. В свойствах задачи посмотрите время последнего запуска и результат; в журнале скрипта должны появиться новые строки, а в out/backups — результат нужной операции. Кнопка «Выполнить» проверяет действие вручную, но не наступление триггера.

23

### Постоянная периодичность

В существующей задаче замените однократный триггер ежедневным, еженедельным или другим из своей карточки. Сохраните задачу и её XML через «Экспорт». Факт выполнения подтверждается вместе: временем запуска, кодом, новой записью скрипта и результатом. Код 0 при отсутствии ожидаемого файла или отчёта означает, что проверка ещё не завершена.

Маршрут / SQL Server Express · Windows

# Итог выбранного маршрута

24

В своей карточке найдите функцию операции и её обязательный опыт. Выполните только её, добавьте выбранное индивидуальное дополнение, затем заполните журнал: подключение, права, ручная копия, отдельное восстановление, ручной запуск, контролируемый отказ и действительный запуск планировщика.

Сдаваемая инструкция своего репозитория позволяет повторить запуск с выданными параметрами без пересоздания производственной модели и без чтения другого маршрута. Снимки и результаты отчёта принадлежат своей работе; приведённые ожидаемые значения не заменяют наблюдения.

Маршрут / MySQL 8 · Windows

# Прежде чем копировать команды

1

## Работайте только в этом маршруте

ОС и СУБД должны совпадать с карточкой варианта. Здесь все команды ОС выполняются в: **Windows PowerShell на учебной Windows-машине**. SQL выполняется отдельно в клиенте **mysql.exe**. Адрес клиента на хосте и адрес внутри VM могут различаться: перепишите оба из паспорта учебной среды. Настройка сервера и канал доступа уже подготовлены; установка из другого маршрута не выполняется.

2

## Заполните подстановки

| В образце | В своём варианте |
|---|---|
| pm07_demo | pm07_vNN — имя исходной базы |
| pm07_demo_restore | pm07_vNN_restore — отдельная проверочная база |
| devices, locations, checks | Три имени таблиц из своей карточки |
| input.sql | Скачанный SQL своего варианта, сохранённый под этим рабочим именем |
| ADMIN_PASSWORD, APP_PASSWORD, READ_PASSWORD | Собственные учебные пароли, подставляемые до выполнения SQL |

В SQL сохраняйте обозначение имени таблицы: обратные кавычки вокруг имени для MySQL/MariaDB. Заменяется имя внутри них. Это важно, например, для исходной таблицы groups.

3

### Заполните подстановки · продолжение

Во всех блоках SQL сначала замените длинное имя pm07_demo_restore, затем pm07_demo. Сохраняйте каждый изменённый блок в текстовом редакторе до запуска. Пароль в SQL-строке заключён в одинарные кавычки; внутреннюю одинарную кавычку удваивают. При подготовке профиля клиента учитывайте его собственное экранирование. Пароли не помещаются в снимки отчёта.

Маршрут / MySQL 8 · Windows

# 1. Подключение и исходные данные

4

### Подготовьте рабочую папку

```
New-Item -ItemType Directory -Force C:\Practice07 | Out-Null
Set-Location C:\Practice07
```

Сохраните в этой папке SQL своей карточки под именем input.sql. Если файл скачан на другой компьютер, выполните перенос и проверку файла. Разбор использует pm07_demo; в своих командах укажите имя базы по номеру варианта.

5

## Сначала задайте пути утилит

В PowerShell выполните, подставив пути своего паспорта:

```
$Client = 'C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe'
$Dump = 'C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe'
Test-Path -LiteralPath $Client
Test-Path -LiteralPath $Dump
& $Client -h 127.0.0.1 -P 3306 -u root -p
```

Оба Test-Path должны вернуть True. После последней команды вы находитесь в клиенте SQL, а не в приглашении PowerShell. Если начальная роль называется иначе, замените root до входа.

6

## Войдите начальным администратором

Используйте выданную команду входа в mysql.exe. При парольной TCP-аутентификации пример в терминале: `mysql.exe -h 127.0.0.1 -P 3306 -u root -p`. Если в паспорте указан локальный socket-вход, используйте именно его. Создание базы и всех учётных записей ниже выполняется начальным администратором сервера, а не пользователем admin одной базы.

В отдельной учебной VM вход для выданных клиентов заранее ограничен подготовленным каналом. В примере имена MySQL-учётных записей имеют host-часть %, чтобы одна учебная роль могла пройти локальную и выданную клиентскую проверку. Эта часть — условие сопоставления клиента, а не команда настройки сетевого доступа.

```
SELECT VERSION(), CURRENT_USER(), DATABASE();
CREATE DATABASE pm07_demo CHARACTER SET utf8mb4;
CREATE DATABASE pm07_demo_restore CHARACTER SET utf8mb4;
CREATE USER 'admin'@'%' IDENTIFIED BY 'ADMIN_PASSWORD';
CREATE USER 'app_user'@'%' IDENTIFIED BY 'APP_PASSWORD';
CREATE USER 'readonly'@'%' IDENTIFIED BY 'READ_PASSWORD';
GRANT ALL PRIVILEGES ON pm07_demo.* TO 'admin'@'%';
GRANT ALL PRIVILEGES ON pm07_demo_restore.* TO 'admin'@'%';
GRANT SELECT,INSERT,UPDATE,DELETE ON pm07_demo.* TO 'app_user'@'%';
GRANT SELECT ON pm07_demo.* TO 'readonly'@'%';
SHOW GRANTS FOR 'admin'@'%';
SHOW GRANTS FOR 'app_user'@'%';
SHOW GRANTS FOR 'readonly'@'%';
```

7

## Загрузите выданный SQL от admin

Завершите соединение командой quit и войдите заново как admin в исходную базу. Полный путь к клиенту Windows берётся из паспорта. В консоли SQL выберите свою базу через USE, затем выполните SOURCE с полным путём к input.sql. На Windows путь записывается C:/Practice07/input.sql; на Linux — значение pwd и /input.sql. Это команда клиента SQL, а не строка PowerShell.

GRANT на pm07_demo.* не выдаёт глобальную привилегию CREATE USER. В этом маршруте серверные учётные записи уже созданы начальным администратором; проверка admin относится к своей базе. Дополнительный FLUSH PRIVILEGES после CREATE USER и GRANT не требуется.

8

## Проверьте исходный набор и графический клиент

Подключитесь к исходной базе как admin и выполните для каждой своей предметной таблицы (в примере devices):

```
SELECT COUNT(*) AS row_count, SUM(id) AS id_sum, SUM(amount) AS amount_sum FROM `devices`;
```

Ожидаются 4, 10, 100. Сохраните три результата с именами таблиц. access_probe содержит id=1 и value=10 и в эти три предметных результата не входит.

9

## Проверьте исходный набор и графический клиент · продолжение

На хосте откройте выданный графический клиент. В DBeaver: «База данных» (Database) → «Новое соединение» (New Database Connection) → драйвер своей СУБД → «Далее» (Next); введите адрес и порт проброса **из паспорта хоста**, имя исходной базы, readonly и его пароль → «Тест соединения» (Test Connection) → «Готово» (Finish). Если отключено «Сохранять пароль», введите пароль в появившемся отдельном окне авторизации; поле пароля в основной форме при этом недоступно. Если клиент просит драйвер, используйте выданный заранее настроенный клиент; загрузка неизвестного драйвера не заменяет подготовку среды. Откройте «Редактор SQL» (SQL Editor) → «Open SQL console» (Ctrl+Alt+Enter) и выполните SELECT COUNT(*) для одной своей таблицы. Пошаговый разбор со снимками находится в общем шаге подключения. В SSMS аналогично выбираются выданный сервер и SQL Server Authentication с readonly, затем New Query и своя база. Результат должен совпасть с CLI. 127.0.0.1 внутри VM и на хосте означает разные машины; порт из гостевого примера не переносится в хост автоматически.

# 2. Три отдельных подключения и права

10

## Позитивные опыты

Для каждого опыта создайте новое соединение и проверьте имя текущей роли. Под admin выполните создание и удаление только учебной check_admin. Под app_user измените отдельную строку access_probe и затем верните её в исходное состояние:

```
CREATE TABLE check_admin (id int);
DROP TABLE check_admin;
```

```
INSERT INTO access_probe(id,value) VALUES(9001,20);
UPDATE access_probe SET value=30 WHERE id=9001;
SELECT * FROM access_probe WHERE id=9001;
DELETE FROM access_probe WHERE id=9001;
```

Первый блок — только admin; второй — только app_user. Затем readonly читает SELECT * FROM access_probe. После опыта остаётся ровно исходная строка id=1, value=10.

11

### Негативные опыты

Под readonly отдельно проверьте INSERT в access_probe и DELETE строки id=1. Под app_user отдельно проверьте CREATE TABLE forbidden_app(id int). Ожидается отказ в соответствующем праве. Если действие неожиданно выполнено, зафиксируйте результат и исправьте выдачу прав из начального административного подключения; не объявляйте такую проверку успешной.

Под app_user проверьте создание серверной учётной записи командой именно этой СУБД:

```
CREATE USER 'forbidden_probe'@'%' IDENTIFIED BY 'Probe-Only-7Q9!23';
```

Нужен отказ в полномочиях, а не ошибка пароля, синтаксиса или уже существующего имени. Не проверяйте запрет командой другого движка. Списки системных схем не обязаны выглядеть одинаково; вывод делается по нужной операции.

Маршрут / MySQL 8 · Windows

# 3. Ручная копия и отдельное восстановление

12

### Сохраните исходные значения

После проверки прав повторите COUNT(*), SUM(id), SUM(amount) по трём таблицам и сохраните вывод как rows-before.txt. Зафиксируйте точное имя исходной базы. Первый опыт не требует повреждения этой базы.

13

## Сохраните дамп в точный файл

В терминале ОС вызовите mysqldump.exe, подставив исходную базу. --result-file сохраняет SQL напрямую; в PowerShell не используется перенаправление >, которое в старых версиях меняет кодировку текста. Не добавляйте --databases: в учебном дампе не должно быть переключения обратно на исходное имя базы.

```
& $Dump -h 127.0.0.1 -P 3306 -u admin -p --single-transaction --no-tablespaces --set-gtid-purged=OFF --default-character-set=utf8mb4 --result-file=manual.sql pm07_demo
```

14

## Загрузите дамп только в _restore

Войдите клиентом mysql.exe под admin. В приглашении SQL выберите USE pm07_demo_restore; проверьте SELECT DATABASE(); и выполните SOURCE с полным точным путём к manual.sql. Звёздочки и имя файла с примерной датой здесь не используются. Исходная база при этом не очищается.

Посчитайте SHA-256: sha256sum manual.sql на Linux или Get-FileHash -LiteralPath .\manual.sql -Algorithm SHA256 на Windows. Затем выполните три контрольных запроса в _restore. Если восстанавливается не текст SQL, а архив PostgreSQL, вы выбрали другой маршрут.

15

## Сверьте три предметные таблицы

Запишите rows-after.txt рядом с rows-before.txt. Сопоставьте имя таблицы, количество строк, сумму id и сумму amount; проверьте также одну конкретную запись по id=1. При различии проверьте выбранный файл и его время, список таблиц, базу назначения и первое сообщение восстановления. Только после совпадения переходите к расписанию.

# 4. Операция своего варианта

16

## Как сохранить файлы без лишнего расширения

Откройте Блокнот. Для config.ps1 и run.ps1 выберите «Сохранить как» → «Все файлы», точное имя и кодировку UTF-8 с BOM, совместимую с Windows PowerShell 5.1. Проводник → Вид → Расширения имён файлов позволяет проверить, что не получилось run.ps1.txt. Каждый блок заменяет файл целиком; не копируйте в него тройные обратные кавычки из оформления.

Для профиля создайте папку C:\Practice07\auth командой New-Item -ItemType Directory -Force. Откройте в Блокноте точный файл, указанный ниже, сохраните как «Все файлы» и подставьте свои значения. Парольный профиль — обычный закрытый файл, а не часть отчёта.

17

## Подготовьте параметры и профиль клиента

Создайте config.ps1 в C:\Practice07. Замените NN и пути примерных утилит значениями паспорта. Выполните Test-Path -LiteralPath для каждого пути к exe: ожидается True.

```
$Database = 'pm07_vNN'
$Client = 'C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe'
$Dump = 'C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe'
$Auth = Join-Path $Work 'auth\client.cnf'
```

В auth/client.cnf сохраните секцию [client] и строки user=admin, password=свой_пароль, host=127.0.0.1, port=3306. Файл создаётся в закрытой рабочей папке пользователя задачи. Сначала проверьте подключение --defaults-file без интерактивного пароля. Этот параметр указывается первым аргументом утилиты.

18

## Сохраните каркас run.ps1

Скопируйте блок целиком. В следующем действии заменяется только Invoke-Task.

```
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
$Work = $PSScriptRoot
. (Join-Path $Work 'config.ps1')
foreach ($dir in 'out','backups','logs') {
    New-Item -ItemType Directory -Force -Path (Join-Path $Work $dir) | Out-Null
}
$Log = Join-Path $Work 'logs\run.log'
$Stamp = Get-Date -Format 'yyyyMMdd_HHmmss'
function Write-Log([string]$Level,[string]$Message) {
    "$(Get-Date -Format o) [$Level] $Message" | Add-Content -LiteralPath $Log -
Encoding UTF8
}

function Invoke-Db([string]$Query) {
    & $Client "--defaults-file=$Auth" --batch --raw --skip-column-names "--
database=$Database" "--execute=$Query"
    if ($LASTEXITCODE -ne 0) { throw "mysql exit=$LASTEXITCODE" }
}
function Backup-Db([string]$File) {
    & $Dump "--defaults-file=$Auth" --single-transaction --no-tablespaces --set-
gtid-purged=OFF --default-character-set=utf8mb4 "--result-file=$File" $Database
```

19

## Сохраните каркас run.ps1 · продолжение

```
    if ($LASTEXITCODE -ne 0) { throw "mysqldump exit=$LASTEXITCODE" }
}

function Invoke-Task {
    throw 'Замените эту функцию целиком функцией своей карточки'
}
try {
    Write-Log INFO "START db=$Database"
    Invoke-Task
    Write-Log INFO 'END result=OK'
    exit 0
} catch {
    Write-Log ERROR $_.Exception.Message
    exit 1
}
```

20

## Ручной запуск и проверка отказа

Сохраните run.ps1 и config.ps1 в C:\Practice07. Замените Invoke-Task целиком функцией своей карточки. Если каркас ещё содержит throw с указанием заменить функцию, операция не выполнена. Сначала используйте ту же учётную запись Windows, от которой затем будет запускаться задача.

```
powershell.exe -NoProfile -File C:\Practice07\run.ps1
$LASTEXITCODE
Get-Content -LiteralPath C:\Practice07\logs\run.log -Tail 8
Get-ChildItem -LiteralPath C:\Practice07\out,C:\Practice07\backups
```

Ожидаются код 0, новая START/END и результат своей операции. Сохраните отдельные run-test.ps1 и config-test.ps1; в копии скрипта измените только имя подключаемого конфига. В тестовом конфиге задайте несуществующую базу, а для файловой операции — несуществующий исходный путь. Ожидаются ERROR и ненулевой код. Рабочий конфиг с паролем не перезаписывается. Если политика учебной машины запрещает локальные скрипты, используйте выданный разрешённый учебный профиль; общий ExecutionPolicy не меняется ради примера.

21

## Создание однократной задачи

Откройте Пуск → Планировщик заданий → Создать задачу. На вкладке «Общие» оставьте ту же учётную запись Windows, которая прошла ручную проверку. Для учебного опыта выберите выполнение при входе этого пользователя в систему; SYSTEM не подставляется вместо него.

На вкладке «Триггеры» нажмите «Создать», выберите «По расписанию» → «Однократно» и время на несколько минут позже текущего. На вкладке «Действия» выберите «Запуск программы» и заполните поля раздельно:

| Поле | Значение |
| --- | --- |
| Программа | C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe |
| Аргументы | -NoProfile -File C:\Practice07\run.ps1 |
| Рабочая папка | C:\Practice07 |

22

### Создание однократной задачи · продолжение

Сохраните задачу с именем PM07_VNN, заменив NN номером. Запишите назначенное время и дождитесь его. В свойствах задачи посмотрите время последнего запуска и результат; в журнале скрипта должны появиться новые строки, а в out/backups — результат нужной операции. Кнопка «Выполнить» проверяет действие вручную, но не наступление триггера.

23

### Постоянная периодичность

В существующей задаче замените однократный триггер ежедневным, еженедельным или другим из своей карточки. Сохраните задачу и её XML через «Экспорт». Факт выполнения подтверждается вместе: временем запуска, кодом, новой записью скрипта и результатом. Код 0 при отсутствии ожидаемого файла или отчёта означает, что проверка ещё не завершена.

Маршрут / MySQL 8 · Windows

# Итог выбранного маршрута

24

В своей карточке найдите функцию операции и её обязательный опыт. Выполните только её, добавьте выбранное индивидуальное дополнение, затем заполните журнал: подключение, права, ручная копия, отдельное восстановление, ручной запуск, контролируемый отказ и действительный запуск планировщика.

Сдаваемая инструкция своего репозитория позволяет повторить запуск с выданными параметрами без пересоздания производственной модели и без чтения другого маршрута. Снимки и результаты отчёта принадлежат своей работе; приведённые ожидаемые значения не заменяют наблюдения.

Маршрут / PostgreSQL 16 · Windows

# Прежде чем копировать команды

1

## Работайте только в этом маршруте

ОС и СУБД должны совпадать с карточкой варианта. Здесь все команды ОС выполняются в: **Windows PowerShell на учебной Windows-машине**. SQL выполняется отдельно в клиенте **psql.exe**. Адрес клиента на хосте и адрес внутри VM могут различаться: перепишите оба из паспорта учебной среды. Настройка сервера и канал доступа уже подготовлены; установка из другого маршрута не выполняется.

2

## Заполните подстановки

| В образце | В своём варианте |
|---|---|
| pm07_demo | pm07_vNN — имя исходной базы |
| pm07_demo_restore | pm07_vNN_restore — отдельная проверочная база |
| devices, locations, checks | Три имени таблиц из своей карточки |
| input.sql | Скачанный SQL своего варианта, сохранённый под этим рабочим именем |
| ADMIN_PASSWORD, APP_PASSWORD, READ_PASSWORD | Собственные учебные пароли, подставляемые до выполнения SQL |

В SQL сохраняйте обозначение имени таблицы: двойные кавычки вокруг имени для PostgreSQL. Заменяется имя внутри них. Это важно, например, для исходной таблицы groups.

3

### Заполните подстановки · продолжение

Во всех блоках SQL сначала замените длинное имя pm07_demo_restore, затем pm07_demo. Сохраняйте каждый изменённый блок в текстовом редакторе до запуска. Пароль в SQL-строке заключён в одинарные кавычки; внутреннюю одинарную кавычку удваивают. При подготовке профиля клиента учитывайте его собственное экранирование. Пароли не помещаются в снимки отчёта.

Маршрут / PostgreSQL 16 · Windows

# 1. Подключение и исходные данные

4

### Подготовьте рабочую папку

```
New-Item -ItemType Directory -Force C:\Practice07 | Out-Null
Set-Location C:\Practice07
```

Сохраните в этой папке SQL своей карточки под именем input.sql. Если файл скачан на другой компьютер, выполните перенос и проверку файла. Разбор использует pm07_demo; в своих командах укажите имя базы по номеру варианта.

5

## Сначала задайте путь клиента

В PowerShell задайте $Client перед его первым использованием:

```
$Client = 'C:\Program Files\PostgreSQL\16\bin\psql.exe'
$Dump = 'C:\Program Files\PostgreSQL\16\bin\pg_dump.exe'
$Restore = 'C:\Program Files\PostgreSQL\16\bin\pg_restore.exe'
Test-Path -LiteralPath $Client
```

Ожидается True; если пути паспорта отличаются, сначала подставьте их. Команда входа начального администратора: & $Client -h 127.0.0.1 -p 5432 -U postgres -d postgres.

6

## Войдите начальным администратором

Откройте psql из выданного каталога клиента. Подключитесь к базе postgres от начальной роли postgres; если роль называется иначе, используйте паспорт. Пример параметров: host 127.0.0.1, port 5432. После запроса введите пароль интерактивно. В клиенте выполните следующий SQL после подстановок:

```
SELECT version(), current_user, current_database();
CREATE ROLE admin LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE PASSWORD
'ADMIN_PASSWORD';
CREATE ROLE app_user LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE PASSWORD
'APP_PASSWORD';
CREATE ROLE readonly LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE PASSWORD
'READ_PASSWORD';
CREATE DATABASE pm07_demo OWNER admin TEMPLATE template0 ENCODING 'UTF8';
CREATE DATABASE pm07_demo_restore OWNER admin TEMPLATE template0 ENCODING
'UTF8';
```

7

## Загрузите выданный SQL от admin

Завершите текущее соединение командой \q. Подключитесь admin к своей исходной базе. Пример команды в терминале ОС:

```
& $Client -h 127.0.0.1 -p 5432 -U admin -d pm07_demo -v ON_ERROR_STOP=1
```

Перед этой командой задайте $Client полным путём к psql.exe из паспорта, например C:\Program Files\PostgreSQL\16\bin\psql.exe. Переменная с примерным несуществующим путём не заменяет установленную утилиту.

В приглашении psql выполните \i и полный путь к input.sql. На Windows используйте путь C:/Practice07/input.sql; на Linux — путь, показанный pwd, с /input.sql в конце. Затем выполните:

```
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO app_user, readonly;
GRANT SELECT,INSERT,UPDATE,DELETE ON ALL TABLES IN SCHEMA public TO app_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;
GRANT USAGE,SELECT ON ALL SEQUENCES IN SCHEMA public TO app_user;
ALTER DEFAULT PRIVILEGES FOR ROLE admin IN SCHEMA public
  GRANT SELECT,INSERT,UPDATE,DELETE ON TABLES TO app_user;
ALTER DEFAULT PRIVILEGES FOR ROLE admin IN SCHEMA public
  GRANT SELECT ON TABLES TO readonly;
```

8

## Загрузите выданный SQL от admin · продолжение

Таблицы создаёт admin, поэтому и правила для будущих таблиц заданы для этой роли. CREATEROLE и SUPERUSER ему для операций своей базы не выдаются. Служебный postgresql.service иногда показывает active (exited); работоспособность конкретного кластера проверяется реальным подключением и SELECT version(), а не одной строкой агрегирующей службы.

9

## Проверьте исходный набор и графический клиент

Подключитесь к исходной базе как admin и выполните для каждой своей предметной таблицы (в примере devices):

```
SELECT COUNT(*) AS row_count, SUM(id) AS id_sum, SUM(amount) AS amount_sum FROM "devices";
```

Ожидаются 4, 10, 100. Сохраните три результата с именами таблиц. access_probe содержит id=1 и value=10 и в эти три предметных результата не входит.

10

## Проверьте исходный набор и графический клиент · продолжение

На хосте откройте выданный графический клиент. В DBeaver: «База данных» (Database) → «Новое соединение» (New Database Connection) → драйвер своей СУБД → «Далее» (Next); введите адрес и порт проброса **из паспорта хоста**, имя исходной базы, readonly и его пароль → «Тест соединения» (Test Connection) → «Готово» (Finish). Если отключено «Сохранять пароль», введите пароль в появившемся отдельном окне авторизации; поле пароля в основной форме при этом недоступно. Если клиент просит драйвер, используйте выданный заранее настроенный клиент; загрузка неизвестного драйвера не заменяет подготовку среды. Откройте «Редактор SQL» (SQL Editor) → «Open SQL console» (Ctrl+Alt+Enter) и выполните SELECT COUNT(*) для одной своей таблицы. Пошаговый разбор со снимками находится в общем шаге подключения. В SSMS аналогично выбираются выданный сервер и SQL Server Authentication с readonly, затем New Query и своя база. Результат должен совпасть с CLI. 127.0.0.1 внутри VM и на хосте означает разные машины; порт из гостевого примера не переносится в хост автоматически.

# 2. Три отдельных подключения и права

11

## Позитивные опыты

Для каждого опыта создайте новое соединение и проверьте имя текущей роли. Под admin выполните создание и удаление только учебной check_admin. Под app_user измените отдельную строку access_probe и затем верните её в исходное состояние:

```
CREATE TABLE check_admin (id int);
DROP TABLE check_admin;
```

```
INSERT INTO access_probe(id,value) VALUES(9001,20);
UPDATE access_probe SET value=30 WHERE id=9001;
SELECT * FROM access_probe WHERE id=9001;
DELETE FROM access_probe WHERE id=9001;
```

Первый блок — только admin; второй — только app_user. Затем readonly читает SELECT * FROM access_probe. После опыта остаётся ровно исходная строка id=1, value=10.

12

### Негативные опыты

Под readonly отдельно проверьте INSERT в access_probe и DELETE строки id=1. Под app_user отдельно проверьте CREATE TABLE forbidden_app(id int). Ожидается отказ в соответствующем праве. Если действие неожиданно выполнено, зафиксируйте результат и исправьте выдачу прав из начального административного подключения; не объявляйте такую проверку успешной.

Под app_user проверьте создание серверной учётной записи командой именно этой СУБД:

```
CREATE ROLE forbidden_probe LOGIN;
```

Нужен отказ в полномочиях, а не ошибка пароля, синтаксиса или уже существующего имени. Не проверяйте запрет командой другого движка. Списки системных схем не обязаны выглядеть одинаково; вывод делается по нужной операции.

Маршрут / PostgreSQL 16 · Windows

# 3. Ручная копия и отдельное восстановление

13

### Сохраните исходные значения

После проверки прав повторите COUNT(*), SUM(id), SUM(amount) по трём таблицам и сохраните вывод как rows-before.txt. Зафиксируйте точное имя исходной базы. Первый опыт не требует повреждения этой базы.

14

## Создайте архив и восстановите пустую копию

В терминале ОС выполните pg_dump от admin с запросом пароля. Полные пути Windows к pg_dump.exe и pg_restore.exe берутся из того же установленного комплекта, что и psql. В примере показаны аргументы утилит; на Windows перед путём в кавычках ставится оператор &.

```
& $Dump -h 127.0.0.1 -p 5432 -U admin -d pm07_demo -Fc -f manual.dump
if ($LASTEXITCODE -ne 0) { throw 'pg_dump failed' }
& $Restore --list manual.dump
& $Restore -h 127.0.0.1 -p 5432 -U admin -d pm07_demo_restore --no-owner --no-
privileges --exit-on-error manual.dump
if ($LASTEXITCODE -ne 0) { throw 'pg_restore failed' }
```

Восстановление направлено в уже созданную пустую _restore; --create не используется. Вход в другую базу того же сервера выполняется отдельным подключением. Запись вида INSERT INTO исходная.public.table SELECT FROM другая_база.public.table не является переносом между базами PostgreSQL.

Посчитайте SHA-256 manual.dump: sha256sum manual.dump на Linux или Get-FileHash -LiteralPath .\manual.dump -Algorithm SHA256 на Windows. Сохраните значение. Подключитесь к _restore и выполните три контрольных запроса. Список архива не заменяет это сравнение.

15

## Проверка другого архива

Для нового или частичного архива используйте ещё одну пустую проверочную базу. Войдите начальной ролью postgres к базе postgres и выполните после замены имени:

```
CREATE DATABASE pm07_demo_restore2 OWNER admin TEMPLATE template0 ENCODING
'UTF8';
```

Первую _restore и исходную базу сохраните. Завершите это соединение. В Windows PowerShell восстановите новый архив от admin; его точный путь заменяет manual.dump:

```
& $Restore -h 127.0.0.1 -p 5432 -U admin -d pm07_demo_restore2 --no-owner --no-
privileges --exit-on-error manual.dump
if ($LASTEXITCODE -ne 0) { throw 'pg_restore failed' }
```

При повторе начальный администратор выделяет следующее уникальное пустое имя. Имя _restore2 используется только как назначение; непустая база без объяснения не очищается. Для частичной копии сравните только включённые таблицы и подтвердите отсутствие остальных.

16

## Сверьте три предметные таблицы

Запишите rows-after.txt рядом с rows-before.txt. Сопоставьте имя таблицы, количество строк, сумму id и сумму amount; проверьте также одну конкретную запись по id=1. При различии проверьте выбранный файл и его время, список таблиц, базу назначения и первое сообщение восстановления. Только после совпадения переходите к расписанию.

# 4. Операция своего варианта

17

## Как сохранить файлы без лишнего расширения

Откройте Блокнот. Для config.ps1 и run.ps1 выберите «Сохранить как» → «Все файлы», точное имя и кодировку UTF-8 с BOM, совместимую с Windows PowerShell 5.1. Проводник → Вид → Расширения имён файлов позволяет проверить, что не получилось run.ps1.txt. Каждый блок заменяет файл целиком; не копируйте в него тройные обратные кавычки из оформления.

Для профиля создайте папку C:\Practice07\auth командой New-Item -ItemType Directory -Force. Откройте в Блокноте точный файл, указанный ниже, сохраните как «Все файлы» и подставьте свои значения. Парольный профиль — обычный закрытый файл, а не часть отчёта.

18

## Подготовьте параметры и профиль клиента

Создайте config.ps1 в C:\Practice07. Замените NN и пути примерных утилит значениями паспорта. Выполните Test-Path -LiteralPath для каждого пути к exe: ожидается True.

```
$Database = 'pm07_vNN'
$DbHost = '127.0.0.1'
$DbPort = '5432'
$Client = 'C:\Program Files\PostgreSQL\16\bin\psql.exe'
$Dump = 'C:\Program Files\PostgreSQL\16\bin\pg_dump.exe'
$env:PGPASSFILE = Join-Path $Work 'auth\pgpass.conf'
```

В auth/pgpass.conf сохраните host:port:database:admin:свой_пароль. Этот файл принадлежит тому же пользователю Windows, что и задача. Символы : и обратный слеш в значениях экранируются обратным слешем. Права доступа к папке не предоставляются посторонним пользователям. Проверьте подключение psql -w без запроса пароля до создания расписания.

19

## Сохраните каркас run.ps1

Скопируйте блок целиком. В следующем действии заменяется только Invoke-Task.

```
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
$Work = $PSScriptRoot
. (Join-Path $Work 'config.ps1')
foreach ($dir in 'out','backups','logs') {
    New-Item -ItemType Directory -Force -Path (Join-Path $Work $dir) | Out-Null
}
$Log = Join-Path $Work 'logs\run.log'
$Stamp = Get-Date -Format 'yyyyMMdd_HHmmss'
function Write-Log([string]$Level,[string]$Message) {
    "$(Get-Date -Format o) [$Level] $Message" | Add-Content -LiteralPath $Log -
Encoding UTF8
}

function Invoke-Db([string]$Query) {
    & $Client -X -w -h $DbHost -p $DbPort -U admin -d $Database -v
ON_ERROR_STOP=1 -A -t -c $Query
    if ($LASTEXITCODE -ne 0) { throw "psql exit=$LASTEXITCODE" }
}
function Backup-Db([string]$File) {
    & $Dump -w -h $DbHost -p $DbPort -U admin -d $Database -Fc -f $File
    if ($LASTEXITCODE -ne 0) { throw "pg_dump exit=$LASTEXITCODE" }
```

20

## Сохраните каркас run.ps1 · продолжение

```
}

function Invoke-Task {
    throw 'Замените эту функцию целиком функцией своей карточки'
}
try {
    Write-Log INFO "START db=$Database"
    Invoke-Task
    Write-Log INFO 'END result=OK'
    exit 0
} catch {
    Write-Log ERROR $_.Exception.Message
    exit 1
}
```

21

## Ручной запуск и проверка отказа

Сохраните run.ps1 и config.ps1 в C:\Practice07. Замените Invoke-Task целиком функцией своей карточки. Если каркас ещё содержит throw с указанием заменить функцию, операция не выполнена. Сначала используйте ту же учётную запись Windows, от которой затем будет запускаться задача.

```
powershell.exe -NoProfile -File C:\Practice07\run.ps1
$LASTEXITCODE
Get-Content -LiteralPath C:\Practice07\logs\run.log -Tail 8
Get-ChildItem -LiteralPath C:\Practice07\out,C:\Practice07\backups
```

Ожидаются код 0, новая START/END и результат своей операции. Сохраните отдельные run-test.ps1 и config-test.ps1; в копии скрипта измените только имя подключаемого конфига. В тестовом конфиге задайте несуществующую базу, а для файловой операции — несуществующий исходный путь. Ожидаются ERROR и ненулевой код. Рабочий конфиг с паролем не перезаписывается. Если политика учебной машины запрещает локальные скрипты, используйте выданный разрешённый учебный профиль; общий ExecutionPolicy не меняется ради примера.

22

## Создание однократной задачи

Откройте Пуск → Планировщик заданий → Создать задачу. На вкладке «Общие» оставьте ту же учётную запись Windows, которая прошла ручную проверку. Для учебного опыта выберите выполнение при входе этого пользователя в систему; SYSTEM не подставляется вместо него.

На вкладке «Триггеры» нажмите «Создать», выберите «По расписанию» → «Однократно» и время на несколько минут позже текущего. На вкладке «Действия» выберите «Запуск программы» и заполните поля раздельно:

| Поле | Значение |
| --- | --- |
| Программа | C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe |
| Аргументы | -NoProfile -File C:\Practice07\run.ps1 |
| Рабочая папка | C:\Practice07 |

23

### Создание однократной задачи · продолжение

Сохраните задачу с именем PM07_VNN, заменив NN номером. Запишите назначенное время и дождитесь его. В свойствах задачи посмотрите время последнего запуска и результат; в журнале скрипта должны появиться новые строки, а в out/backups — результат нужной операции. Кнопка «Выполнить» проверяет действие вручную, но не наступление триггера.

24

### Постоянная периодичность

В существующей задаче замените однократный триггер ежедневным, еженедельным или другим из своей карточки. Сохраните задачу и её XML через «Экспорт». Факт выполнения подтверждается вместе: временем запуска, кодом, новой записью скрипта и результатом. Код 0 при отсутствии ожидаемого файла или отчёта означает, что проверка ещё не завершена.

Маршрут / PostgreSQL 16 · Windows

# Итог выбранного маршрута

25

В своей карточке найдите функцию операции и её обязательный опыт. Выполните только её, добавьте выбранное индивидуальное дополнение, затем заполните журнал: подключение, права, ручная копия, отдельное восстановление, ручной запуск, контролируемый отказ и действительный запуск планировщика.

Сдаваемая инструкция своего репозитория позволяет повторить запуск с выданными параметрами без пересоздания производственной модели и без чтения другого маршрута. Снимки и результаты отчёта принадлежат своей работе; приведённые ожидаемые значения не заменяют наблюдения.

ПМ.07 / Соадминистрирование

# Разные части окружения

1

СУБД — программа, которая хранит данные и проверяет запросы. Сервер — запущенный процесс СУБД на учебной машине. База — именованный набор объектов внутри сервера. Клиент — программа, из которой отправляется запрос: mysql/mariadb, psql, sqlcmd, DBeaver или SSMS. Создание нового подключения в клиенте не создаёт новый сервер.

ПМ.07 / Соадминистрирование

# Локальное и внешнее подключение

2

127.0.0.1 означает ту машину, на которой запущен клиент. Внутри VM это VM; на хосте — хост. При пробросе портов клиент на хосте использует выданный внешний порт, а СУБД слушает внутренний. Их значения не обязаны совпадать. Успех проверки TCP-порта подтверждает доступность порта, но не правильность SQL-логина и прав.

ПМ.07 / Соадминистрирование

# Паспорт учебной среды

3

До первого действия нужны: ОС, СУБД и версия, адрес и порт изнутри VM, адрес и порт с хоста, начальная учётная запись, рабочая папка, пути клиентских утилит и пользователь планировщика. Для SQL Server дополнительно указывается серверный каталог бэкапов и право записи службы. Неизвестные значения запрашиваются как исходные данные, а не угадываются по соседнему маршруту.

ПМ.07 / Соадминистрирование

# Почему установка вынесена за маршрут

4

Методичка рассчитана на подготовленную среду. Установка ОС и СУБД может потребоваться при её подготовке, но не заменяет проверку доступа, резервирования и расписания. Эта практика даёт обеспечивающие умения администрирования; её серверная установка не объявляется отдельным выполненным экзаменационным заданием.

ПМ.07 / Соадминистрирование

# Готовая учебная схема

5

SQL своей карточки уже содержит минимальные таблицы и строки. Задача здесь — администрирование, поэтому бизнес-модель другого ПМ повторно не проектируется. Поле amount используется как контрольное числовое значение, event_at — для временных условий, link_id — для учебного сопоставления строк. Структура не претендует на полный проект действующей организации.

ПМ.07 / Соадминистрирование

# Уровень разрешения

1

Право задаёт действие и область. Возможность создавать таблицы одной базы не означает возможность создавать учётные записи всего сервера. В этой практике начальный администратор подготавливает пользователей, admin управляет своей базой, app_user выполняет операции с данными, readonly читает.

ПМ.07 / Соадминистрирование

# Особенности СУБД

2

В MySQL и MariaDB в имени учётной записи участвует host-часть: одинаковое имя с разными host — разные записи. Это особенность этих СУБД, а не общее правило для всех программ на Linux. GRANT на имя_базы.* ограничен этой базой; CREATE USER — отдельная серверная операция.

В PostgreSQL роль с LOGIN может входить на сервер. SUPERUSER, CREATEDB и CREATEROLE — отдельные атрибуты, а не обязательные признаки владельца одной базы. Права на схему, существующие таблицы и будущие таблицы проверяются отдельно. Правило ALTER DEFAULT PRIVILEGES относится к объектам, которые создаст указанная роль.

В SQL Server логин относится к серверу, а пользователь — к конкретной базе. Членство в db_owner не превращает SQL-логин в sysadmin. Windows-аккаунт запланированного задания и SQL-логин — также разные сущности.

ПМ.07 / Соадминистрирование

# Как доказать разграничение

3

Для каждой роли нужен отдельный вход и проверка разрешённого действия. Только после него проверяется запрет. Иначе ошибка подключения или опечатка может быть ошибочно принята за правильный запрет. В отчёте называются операция, текущая роль, база и причина отказа.

ПМ.07 / Соадминистрирование

# Видимость и доступ

4

Имена служебных баз и схем могут отображаться по-разному. Их видимость не равна праву изменять данные. В этой работе сравниваются конкретные полномочия и операции своей базы. Несовместимый синтаксис другой СУБД не используется как отрицательный тест.

Все точные команды находятся только в выбранном маршруте. Отдельной альтернативной последовательности SQL в теории нет.

ПМ.07 / Соадминистрирование

# Логическая копия и архив

1

SQL-дамп MySQL/MariaDB содержит команды для восстановления объектов и данных. В PostgreSQL plain SQL читается psql, а формат custom восстанавливается pg_restore. Файл SQL Server .bak обрабатывается BACKUP/RESTORE. Расширение файла само по себе не меняет формат.

ПМ.07 / Соадминистрирование

# Полная и частичная копия

2

Ручная копия всех предметных таблиц нужна каждому варианту. Операция по расписанию может создавать только структуру, выбранные таблицы или другой результат; такая операция не заменяет обязательную полную ручную проверку. Таблица, не включённая в частичный архив, не появится в восстановленной базе автоматически.

ПМ.07 / Соадминистрирование

# Что хранится отдельно

3

Учётные записи сервера и параметры расписания не следует считать автоматически сохранёнными в дампе одной базы. SQL выдачи прав, параметры без паролей и инструкция запуска передаются отдельно. Сам файл копии находится вне каталога данных работающей СУБД.

ПМ.07 / Соадминистрирование

# Проверка файла и проверка данных

4

Ненулевой размер, читаемое оглавление и SHA-256 отвечают на разные вопросы. SHA-256 позволяет сравнить байты файла; он не доказывает, что в архив включены все нужные таблицы. VERIFYONLY и --list дают сведения о копии, но не заменяют запросы к восстановленной базе.

ПМ.07 / Соадминистрирование

# Отдельное восстановление

5

Первый опыт выполняется в _restore. Исходная база сохраняется для сравнения. Контроль включает все три таблицы, COUNT(*), SUM(id), SUM(amount) и конкретную строку. Одинаковое число строк при разных значениях не является полным совпадением данных.

ПМ.07 / Соадминистрирование

# Почему важна точная копия

6

В журнале сохраняются имя базы, точное имя файла и время. Маска со звёздочкой может подобрать несколько файлов. Если восстановлена другая копия, даже успешная команда не подтверждает нужный опыт. В выбранном маршруте имя файла задаётся явно.

ПМ.07 / Соадминистрирование

# Три разных результата

1

Команда выполняет операцию. Скрипт задаёт параметры, проверяет ошибку и сохраняет журнал. Планировщик запускает скрипт в заданное время от конкретного пользователя ОС. Сохранённое расписание ещё не подтверждает, что команда действительно выполнилась.

ПМ.07 / Соадминистрирование

# Что делает каркас

2

Каркас знает рабочую папку и профиль выбранного клиента, создаёт собственные out/backups/logs, записывает START, вызывает ровно одну функцию варианта и при успехе записывает END. При ошибке возвращается ненулевой код. Незаменённая функция-заглушка завершает работу ошибкой, чтобы пустой каркас нельзя было принять за готовый вариант.

ПМ.07 / Соадминистрирование

# Код и фактический результат

3

В Bash ошибка важной команды должна прервать обычную ветвь; в PowerShell внешний exe дополнительно проверяется по LASTEXITCODE. Наличие TRY/CATCH само по себе не превращает любой ненулевой код внешней программы в исключение. Поэтому каркас проверяет его явно.

ПМ.07 / Соадминистрирование

# Контекст пользователя

4

При расписании могут отличаться текущая папка, PATH, парольный профиль и доступ к файлам. Ручной и запланированный опыт выполняются от одной выданной учётной записи. SQL Server пишет BACKUP от своей службы на серверной машине; PowerShell-клиент не становится владельцем этого файла автоматически. Экспортированный профиль одного пользователя Windows не подставляется под SYSTEM.

ПМ.07 / Соадминистрирование

# Как проверить отказ

5

Меняется отдельная копия конфигурации, а не единственный рабочий парольный файл. Неверная база, отсутствующий входной файл или недоступный каталог должны дать понятный ERROR и ненулевой код. После исправления повторяется тот же опыт. «Файл появился» и «в логе написано OK» проверяются вместе с его содержимым.

ПМ.07 / Соадминистрирование

# Как проверить время

6

Сначала назначается ближайший тестовый триггер. Нужно дождаться этого времени и сопоставить запись планировщика, новые строки журнала и результат операции. Ручная кнопка «Выполнить» не подтверждает срабатывание временного триггера. Затем одна тестовая запись заменяется постоянной с частотой своей карточки.

ПМ.07 / Соадминистрирование

# Индивидуальное изменение

7

Выбор операции и дополнения описан в списке областей. Если проверка кода, конфигурация или INFO/ERROR уже присутствует в образце, её простое сохранение не является новым дополнением. Объясните конкретное изменение поведения и проверочный пример.

# Документация команд

1

Методичка содержит обязательные шаги целиком. Эти ссылки служат для уточнения возможностей выбранной версии, а не для поиска пропущенного этапа.

2

**· продолжение**

- MySQL 8.0: GRANT — область привилегий.
- MySQL 8.0: mysqldump — параметры, формат и result-file.
- MariaDB: mariadb-dump — дамп своей СУБД.
- PostgreSQL 16: роли, pg_dump, pg_restore.
- SQL Server: сжатие копий и поддерживаемые редакции.
- SQL Server Express: запуск резервирования по расписанию.